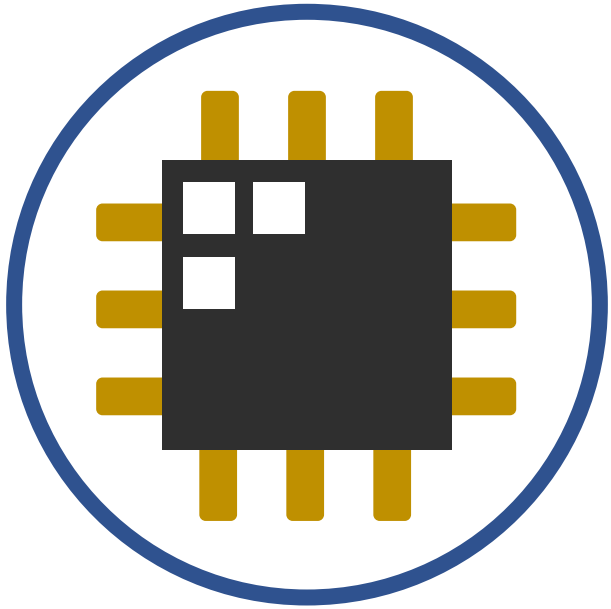
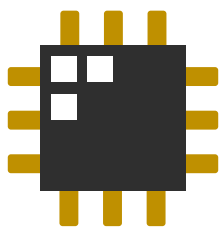




EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 3 – Introduction to Assembly
Architecting a Processor

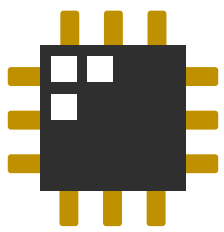


Talking to your computer: Assembly language

- Before we start creating a processor, we should know its *language* first
- Computers use a low-level language called **assembly**.
- Assembly contains **mnemonic codes (instructions)** that specify what the processor should do.
- An instruction specifies the **operation** to perform and the **operands** on which to operate

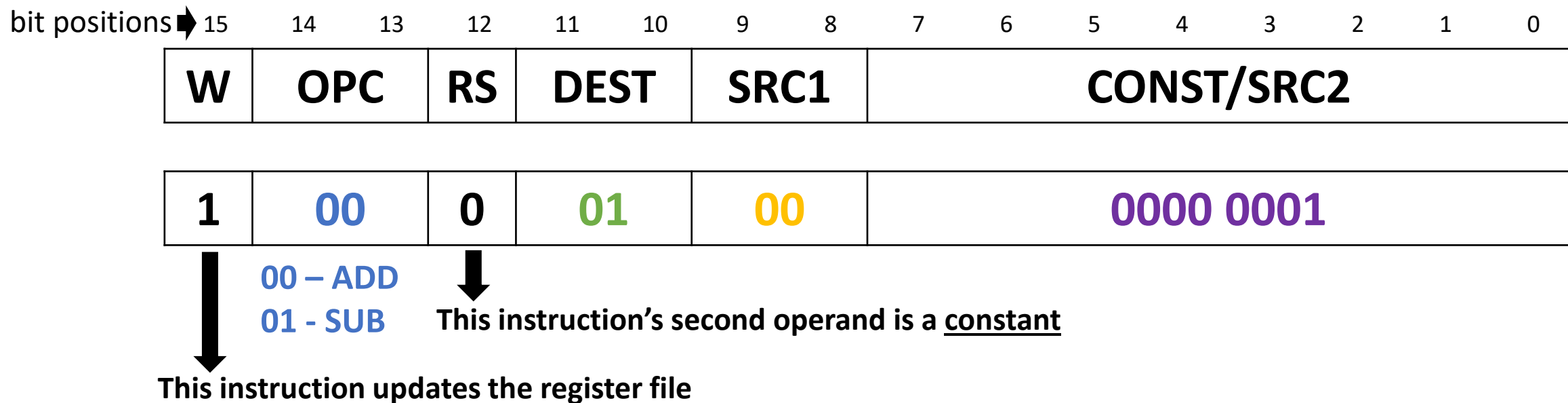
ADD R1, R0, 1

(R1 = R0 + 1)



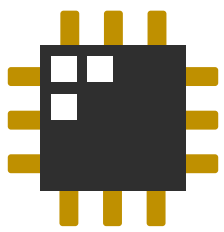
Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



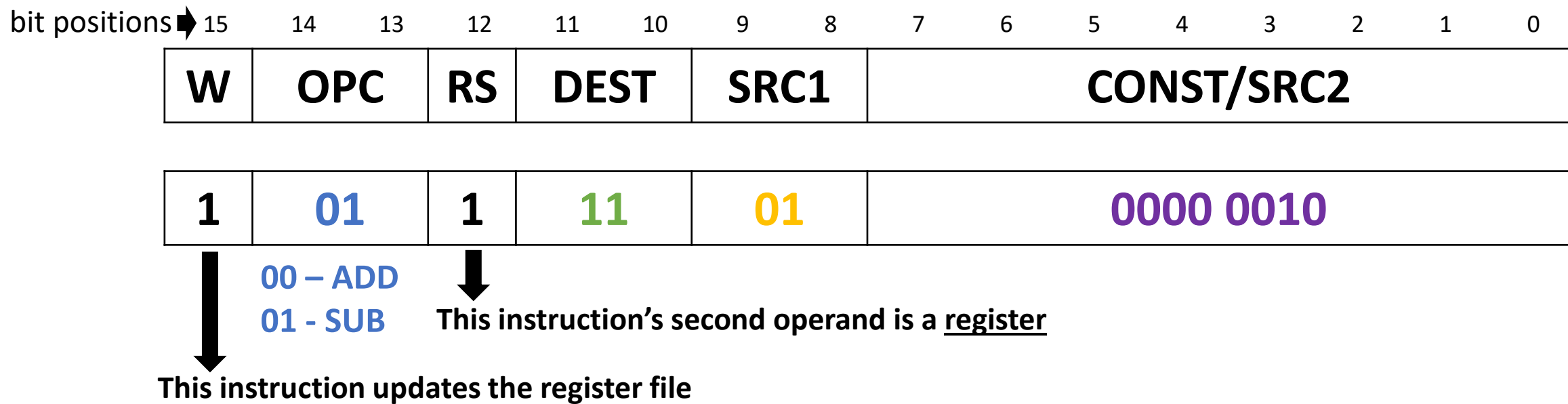
ADD R1, R0, 1

(R1 = R0 + 1) Think of Rx as your variable in a program!



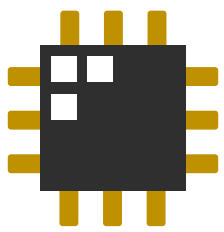
Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



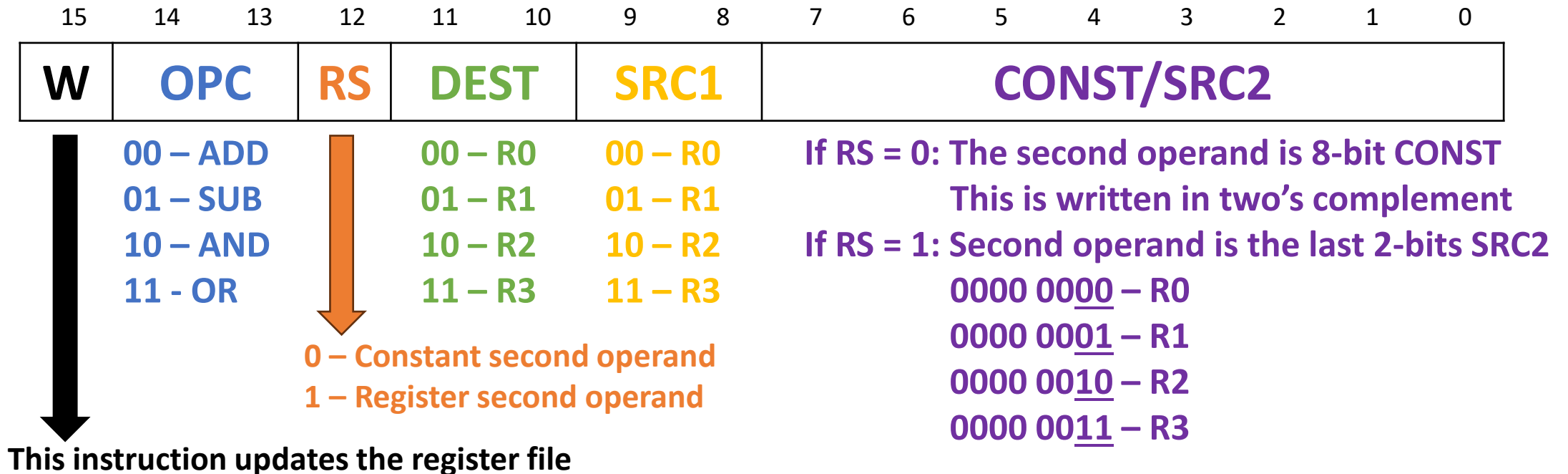
SUB R3, R1, R2

(R3 = R1 – R2) Think of Rx as your variable in a program!

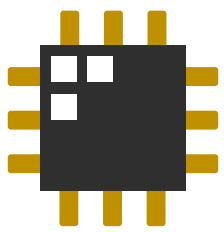


Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



Note: For EEE 113, we will be architecting a custom processor for easy learning 😊
The binary encoding for that custom processor is shown here.



Playing around with instructions

- If we have the following instructions, what would be the contents of the register file every step of the way? (Assume all register values start at 0)

ADDI R3, R0, 113

ADDI R2, R2, 21

SUB R1, R3, R2

ANDI R1, R1, 15

OR R0, R2, R1

Note: We are changing the syntax a bit for easier identification:

ADDI/SUBI/ANDI/ORI* means RS = 0 (the second operand is a constant)

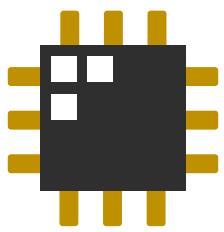
ADD/SUB/AND/OR means RS = 1 (the second operand is a register)

Additional note on AND and OR:

These are bitwise operations.

You'll have to write down the binary bits to be able to perform these operations

*The additional I at the end means *immediate*, computer architecture term for a constant



Playing around with instructions

- If we have the following instructions, what would be the contents of the register file every step of the way? (Assume all register values start at 0)

ADDI R3, R0, 113

R3 = R0 (initially 0) + 113 = 113

ADDI R2, R2, 21

R2 = R2 (initially 0) + 21 = 21

SUB R1, R3, R2

R1 = R3 (113) – R2 (21) = 92

ANDI R1, R1, 15

R1 = R1 (92) bitwise AND 15 = 12

OR R0, R2, R1

R0 = R2 (21) bitwise OR R1 (12) = 29

Note: We are changing the syntax a bit for easier identification:

ADDI/SUBI/ANDI/ORI* means RS = 0 (the second operand is a constant)

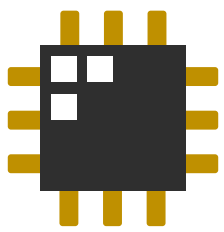
ADD/SUB/AND/OR means RS = 1 (the second operand is a register)

Additional note on AND and OR:

These are bitwise operations.

You'll have to write down the binary bits to be able to perform these operations

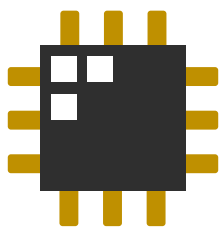
*The additional I at the end means *immediate*, computer architecture term for a constant



Converting instructions to machine code

- For us to run an instruction on the processor, we have to convert the instruction into its binary string (machine code) equivalent.
 - There is dedicated software to do this for actual processors. They are called **assemblers**.
- For now, let's practice becoming the assembler. 😊
 - If you know what the different binary fields are for an instruction, we can do it!

W	OPC	RS	DEST	SRC1	CONST/SRC2
	00 – ADD		00 – R0	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	01 – SUB		01 – R1	01 – R1	If RS = 1: Second operand is the last 2-bits SRC2
	10 – AND		10 – R2	10 – R2	0000 00 <u>00</u> – R0
	11 – OR		11 – R3	11 – R3	0000 00 <u>01</u> – R1
					0000 00 <u>10</u> – R2
This instruction updates the register file					

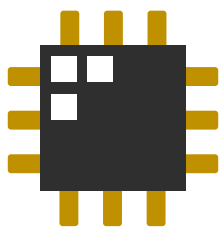


Converting instructions to machine code

- Convert the following instruction into machine code (write the code in hex)

SUBI R3, R1, -31

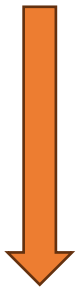
W	OPC	RS	DEST	SRC1	CONST/SRC2
	00 – ADD		00 – R0	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement If RS = 1: Second operand is the last 2-bits SRC2 0000 00 <u>00</u> – R0 0000 00 <u>01</u> – R1 0000 00 <u>10</u> – R2 0000 00 <u>11</u> – R3
	01 – SUB		01 – R1	01 – R1	
	10 – AND		10 – R2	10 – R2	
	11 – OR		11 – R3	11 – R3	
0 – Constant second operand 1 – Register second operand					
This instruction updates the register file					

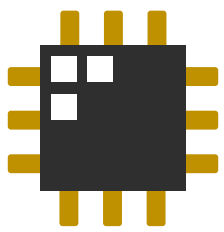


Converting instructions to machine code

- Convert the following instruction into machine code (write the code in hex)

SUBI R3, R1, -31

W	OPC	RS	DEST	SRC1	CONST/SRC2
	00 – ADD		00 – R0	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement If RS = 1: Second operand is the last 2-bits SRC2 0000 00<u>00</u> – R0 0000 00<u>01</u> – R1 0000 00<u>10</u> – R2 0000 00<u>11</u> – R3
	01 – SUB		01 – R1	01 – R1	
	10 – AND		10 – R2	10 – R2	
	11 – OR		11 – R3	11 – R3	
			0 – Constant second operand		
			1 – Register second operand		
This instruction updates the register file					
1	01	0	11	01	1110 0001

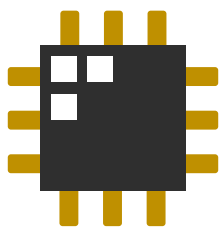


Converting instructions to machine code

- Machine code for instructions is typically written in hex to minimize typo errors

SUBI R3, R1, -31

1	01	0	11	01	1110 0001
----------	-----------	----------	-----------	-----------	------------------



Converting instructions to machine code

- Machine code for instructions is typically written in hex to minimize typo errors

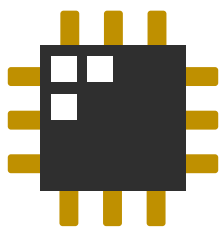
SUBI R3, R1, -31

1	01	0	11	01	1110 0001
---	----	---	----	----	-----------

1010 1101 1110 0001

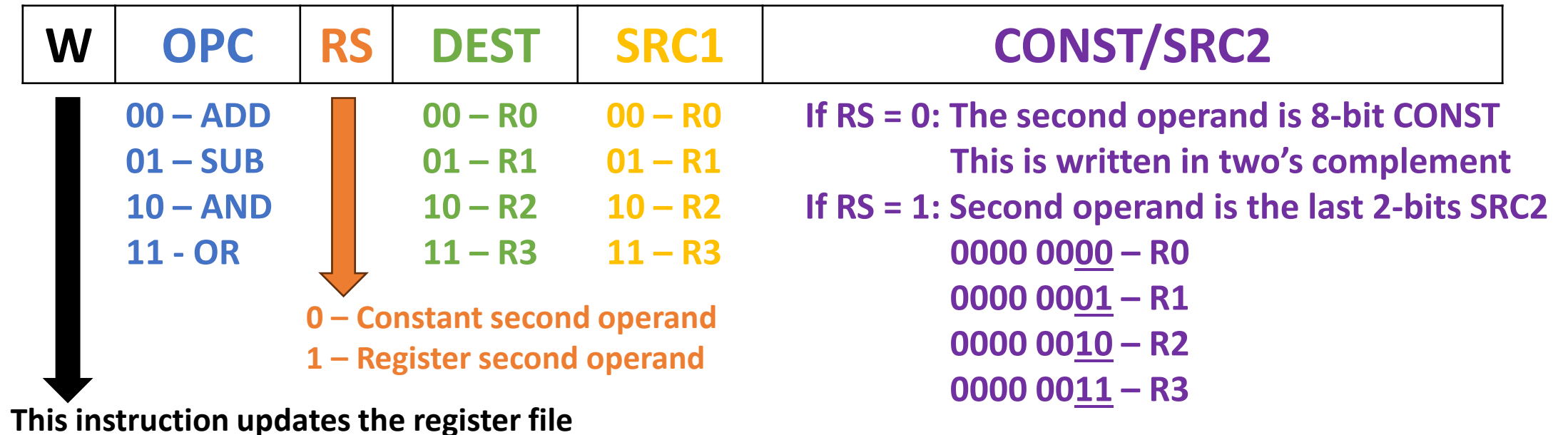
0xADE1

(The equivalent machine code in hex)

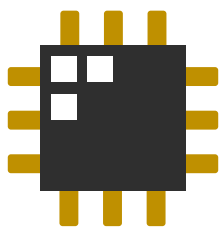


Recap: Instruction encoding

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*

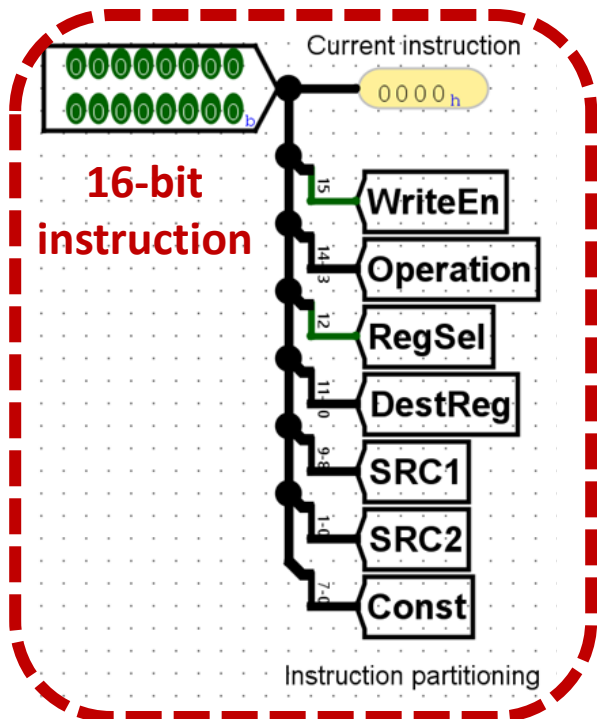
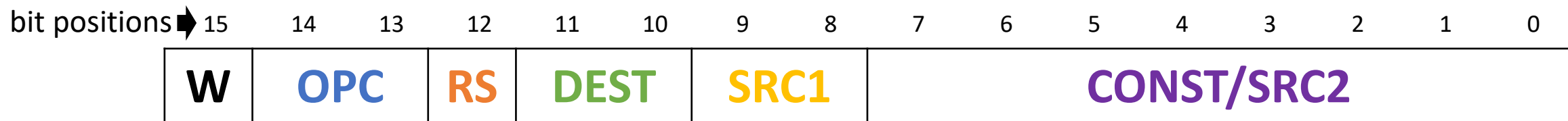


We can now start architecting the processor!



Processor implementation: Step 1

We first need to split the 16-bit instruction into parts



These are bits being partitioned and will be sent out to *control* the processor

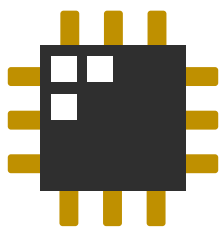
Note:

W = WriteEn

OPC = Operation

RS = RegSel

DEST = DestReg



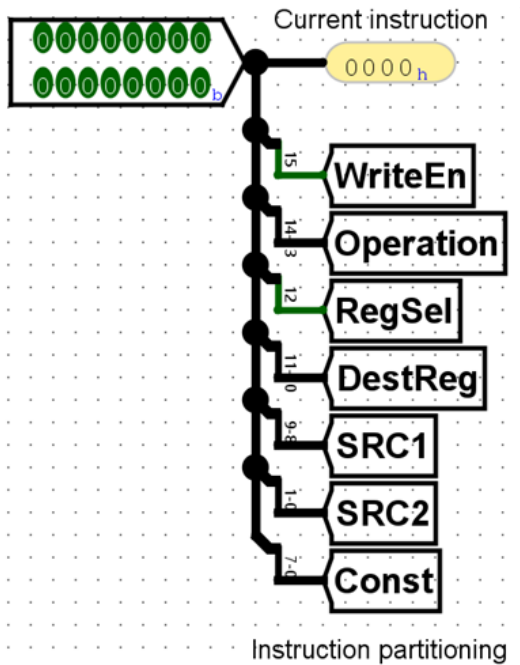
Processor implementation: Step 1

If we are operating using registers R0, R1, R2, R3...

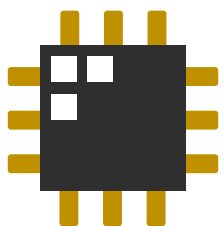
We need to read their register values.

What are we missing?

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

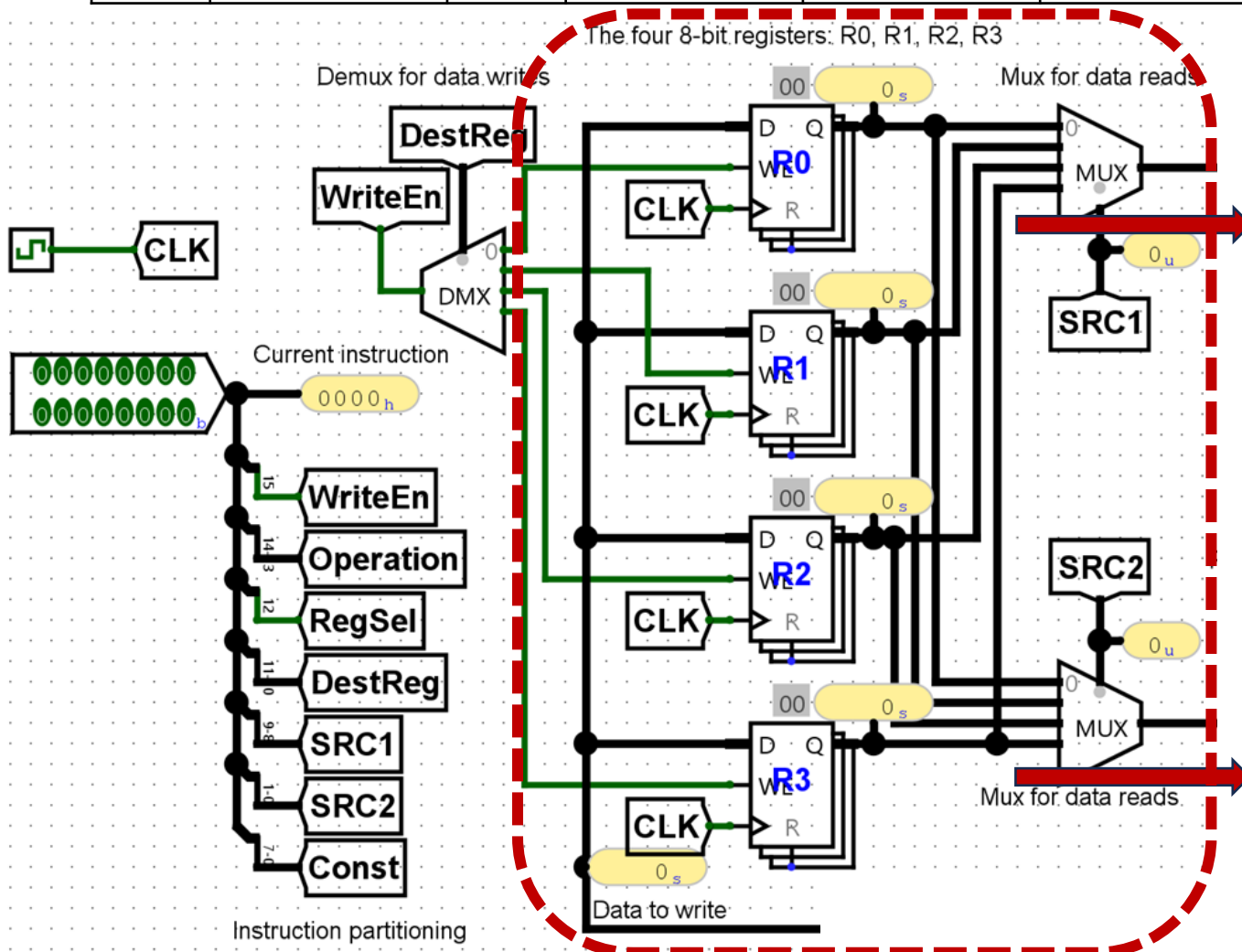






Processor implementation: Step 2

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



Register File!

Multiplexer selects SRC1 (the first operand)

Multiplexer selects SRC2 (the second operand)
...if RS = 1 (The second operand is a register)



The diagram illustrates the internal logic of the ALU stage, showing how an instruction is partitioned into fields and how these fields control the register file and the ALU operations.

Instruction Partitioning: The instruction is divided into several fields, each with a specific bit range:

- WriteEn:** Bit 15 (Write Enable)
- Operation:** Bits 14:3 (ALU Operation)
- RegSel:** Bits 12:12 (Register Selection)
- DestReg:** Bits 11:0 (Destination Register)
- SRC1:** Bits 9:8 (Source Register 1)
- SRC2:** Bits 7:6 (Source Register 2)
- Const:** Bits 7:0 (Constant value)

Register File: The register file contains four 8-bit registers (R0, R1, R2, R3). Each register has a Data (D), Write Enable (WE), and Read (R) port. The registers are initialized to 00h.

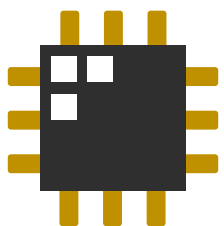
Control Logic: A multiplexer (MUX) selects the destination register (DestReg) for writing. The Write Enable (WriteEn) signal is used to enable writing to the selected register. The Read Enable (R) signal is used to enable reading from the registers.

ALU Operations: The ALU performs operations based on the Operation field. The inputs to the ALU are the outputs of the registers (R0, R1, R2, R3) and the Constant value (Const). The ALU output is connected to the MUX for data reads.

Data Flow: The ALU output is connected to the MUX for data reads, which selects the data to be read from the registers (R0, R1, R2, R3) or the Constant value (Const).

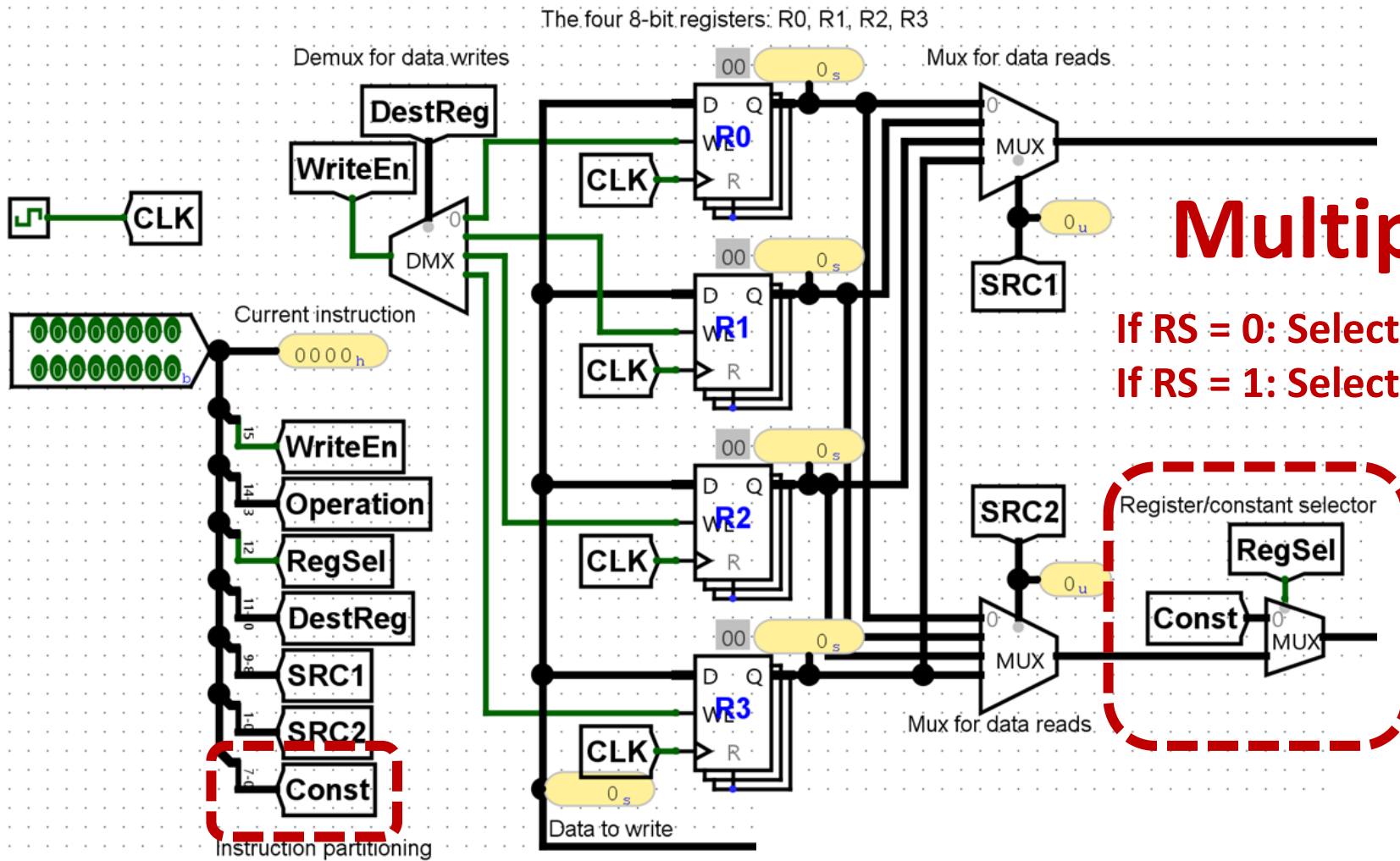
We should select whether we want a register value or a constant...

What else are we missing?



Processor implementation: Step 3

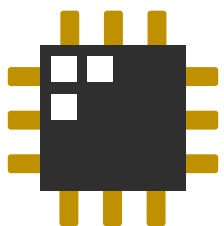
W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



Multiplexer!

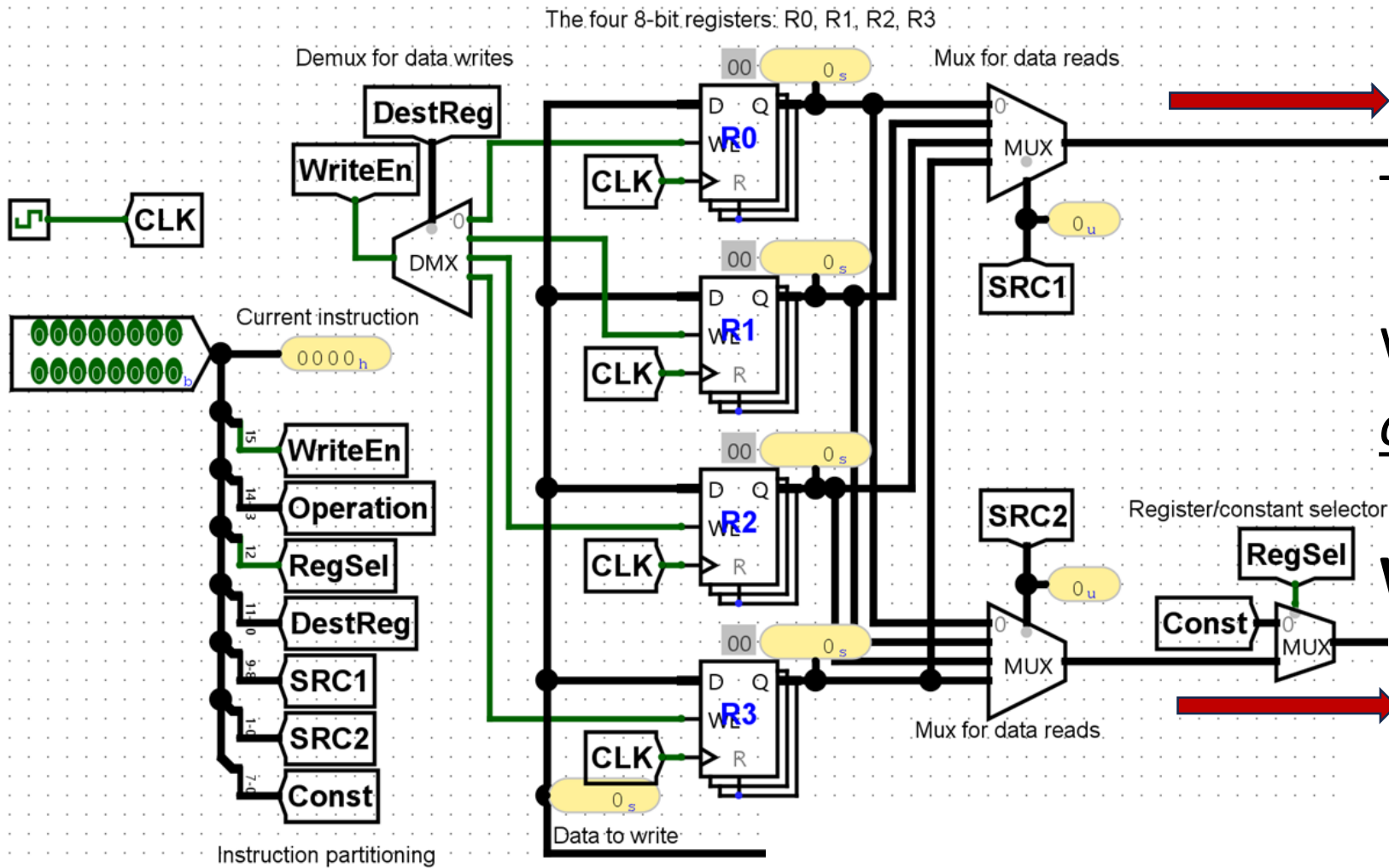
If RS = 0: Select the 8-bit constant as the operand

If RS = 1: Select the output of the register file



Processor implementation: Step 3

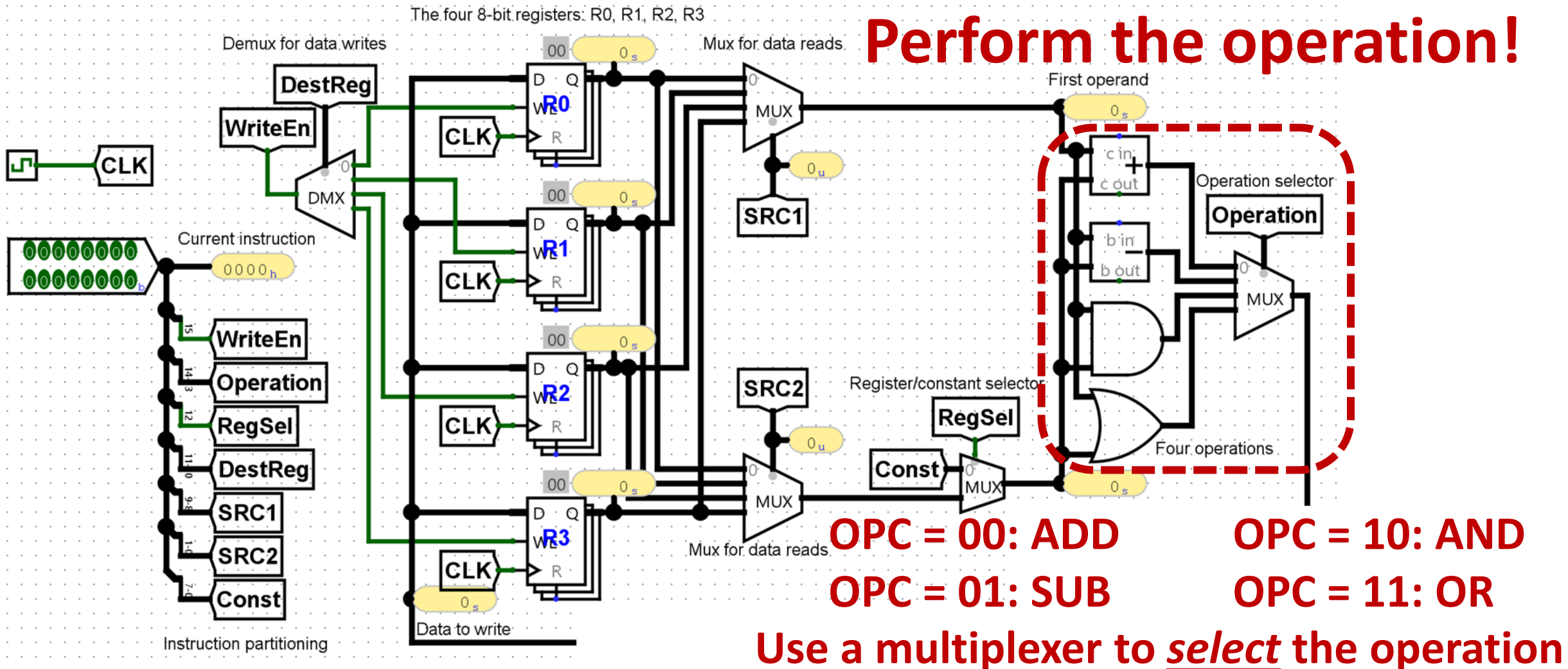
W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



The operands are ready...

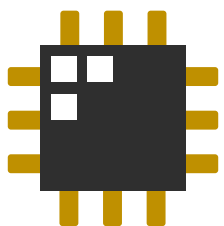
We can now perform the operation!

What are we missing?



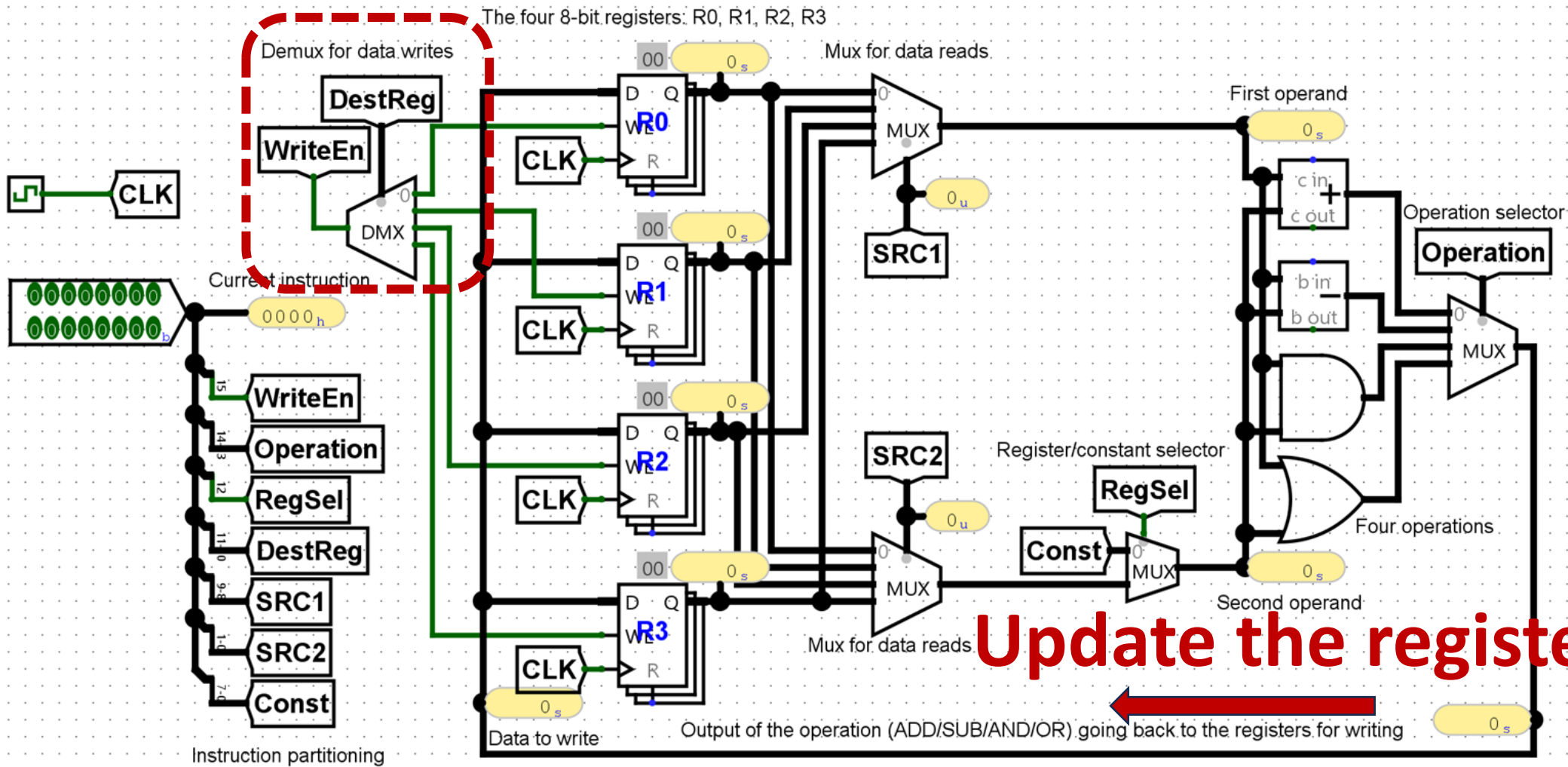


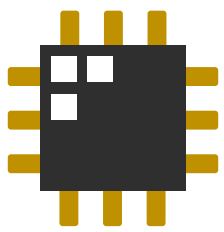
We've done the operation...
What else are we missing?



Processor implementation: Step 5

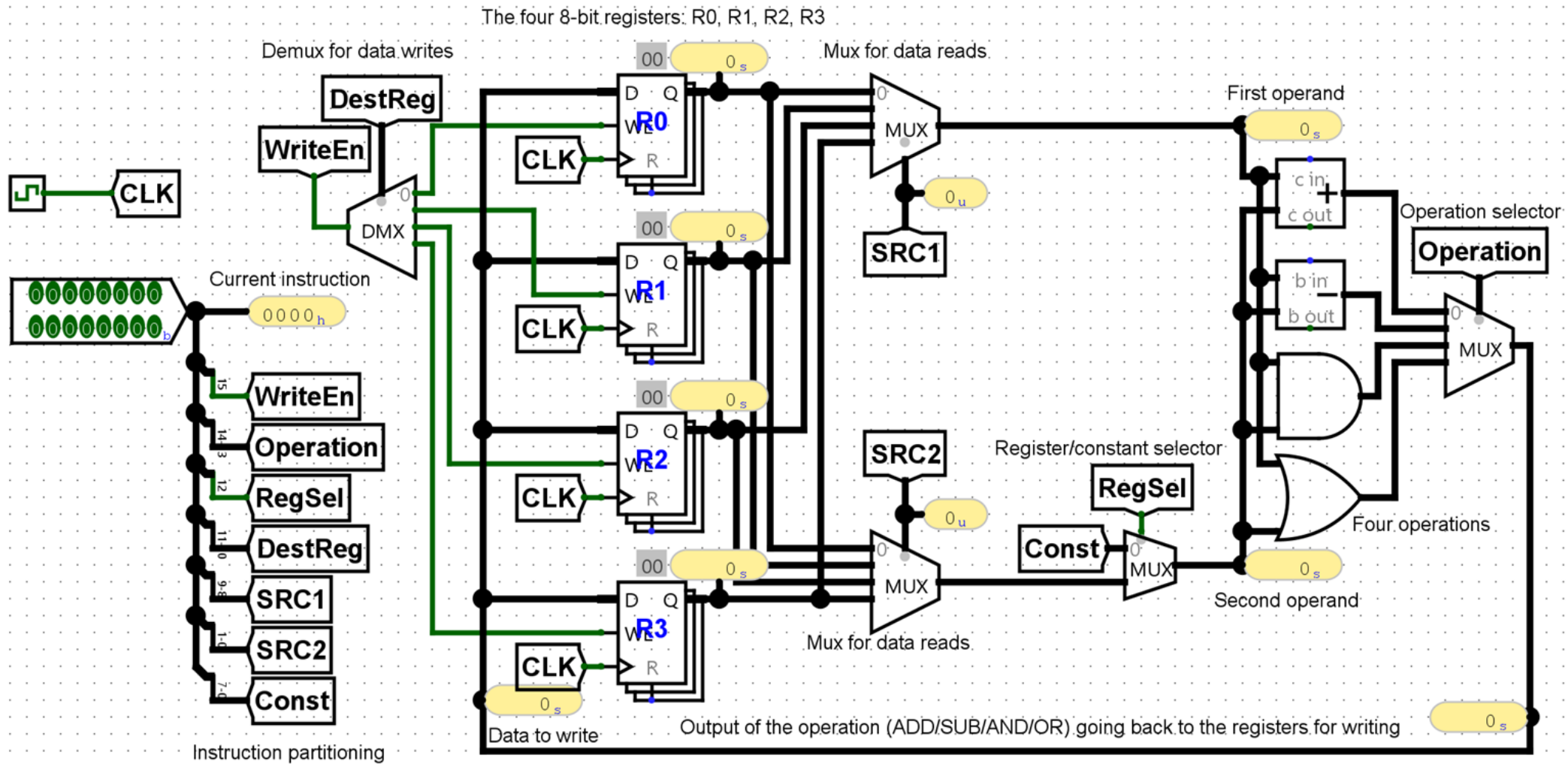
W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

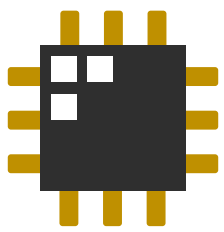




Processor implementation: Version 1

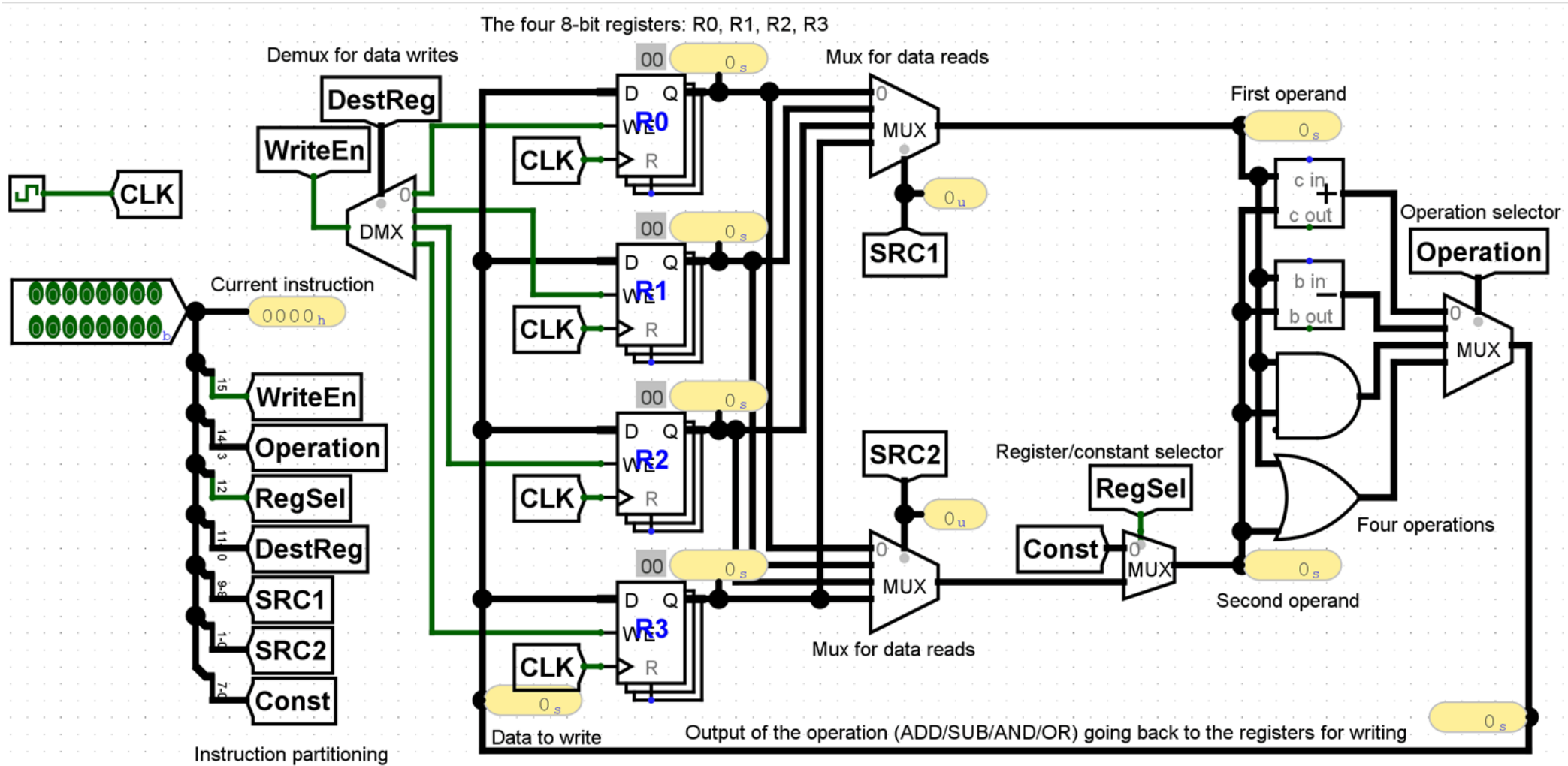
This processor can perform **ADD/SUB/AND/OR** on 4 registers and a constant

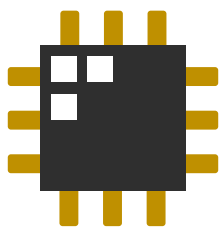




Processor implementation: Version 1

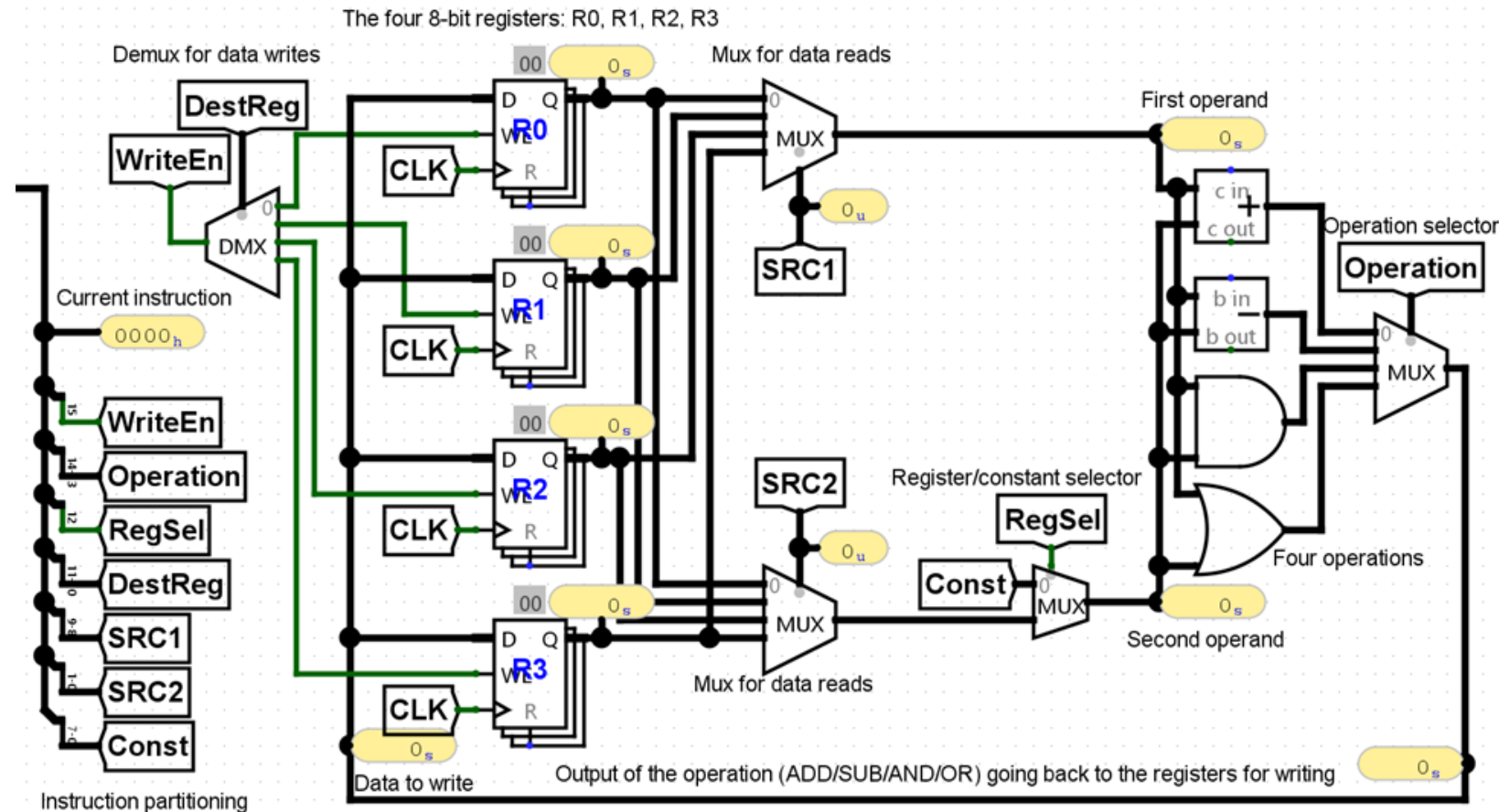
Try it out! SUBI R3, R1, -31 (0xADE1 in hex) (**R3 = R1 (initially 0) – (-31) = 31**)





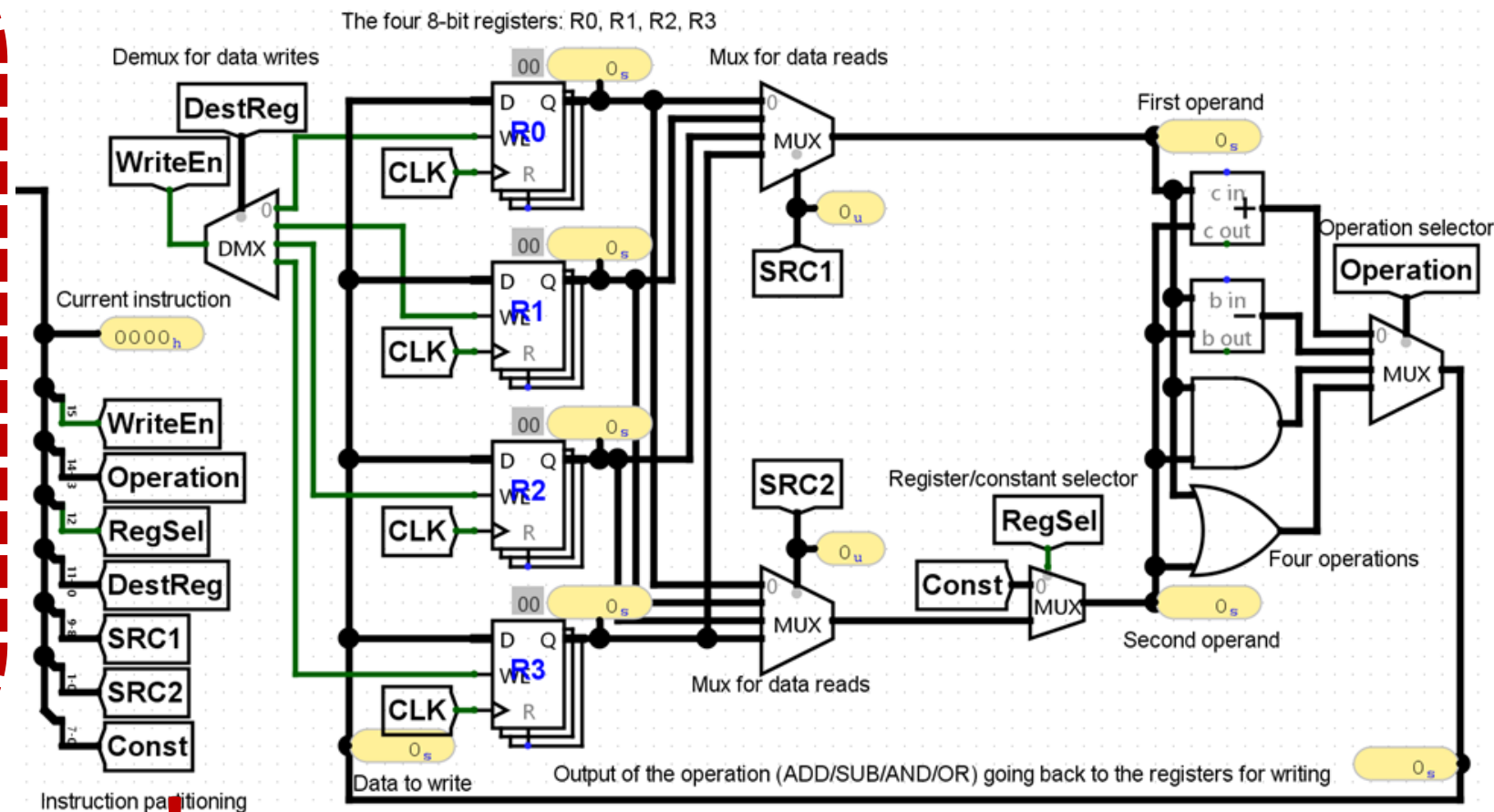
Running multiple instructions

- If we want to run consecutive instructions, we have to **store them in a memory**

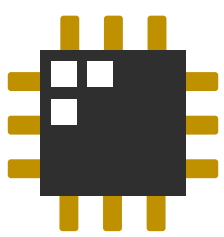




- Recall from the previous lecture...*



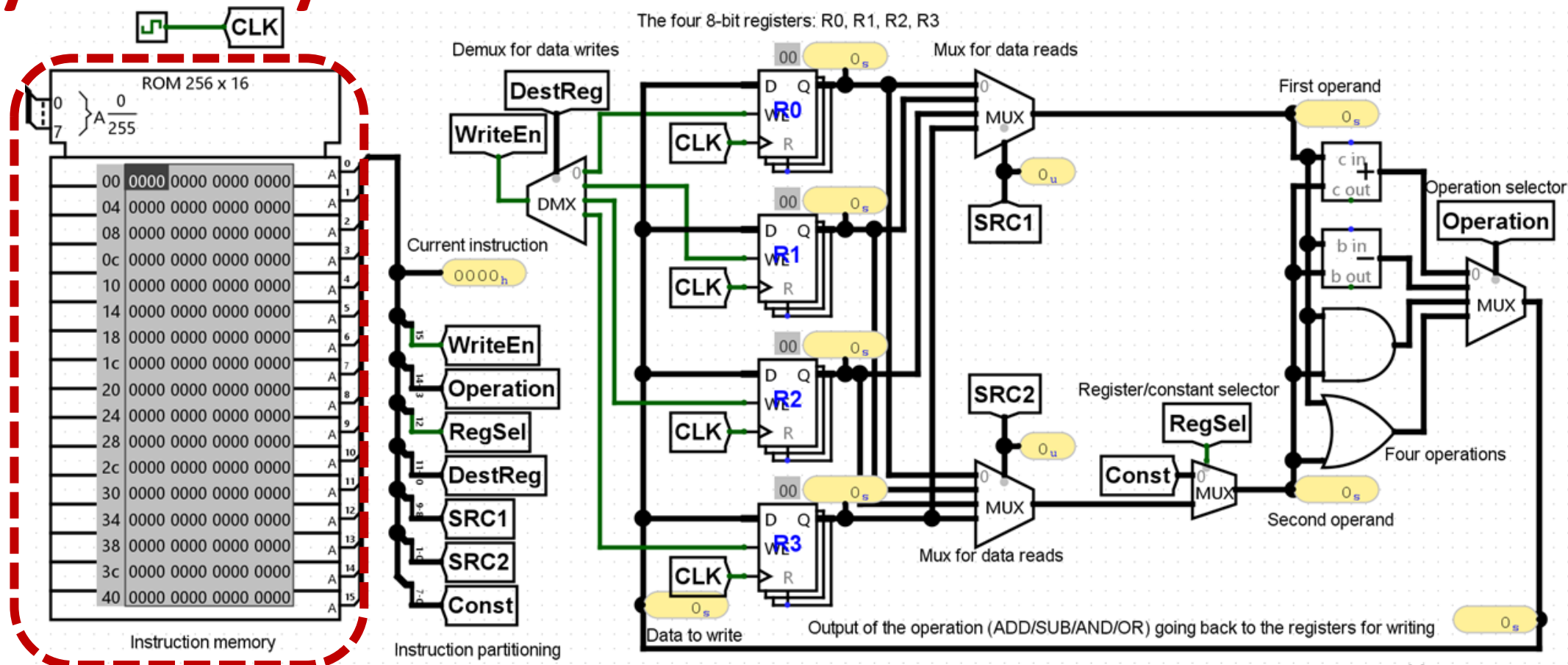
27

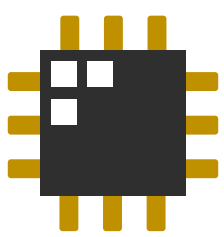


Processor implementation: Step 6

- If we want to run consecutive instructions, we have to **store them in a memory**

Read-only memory!





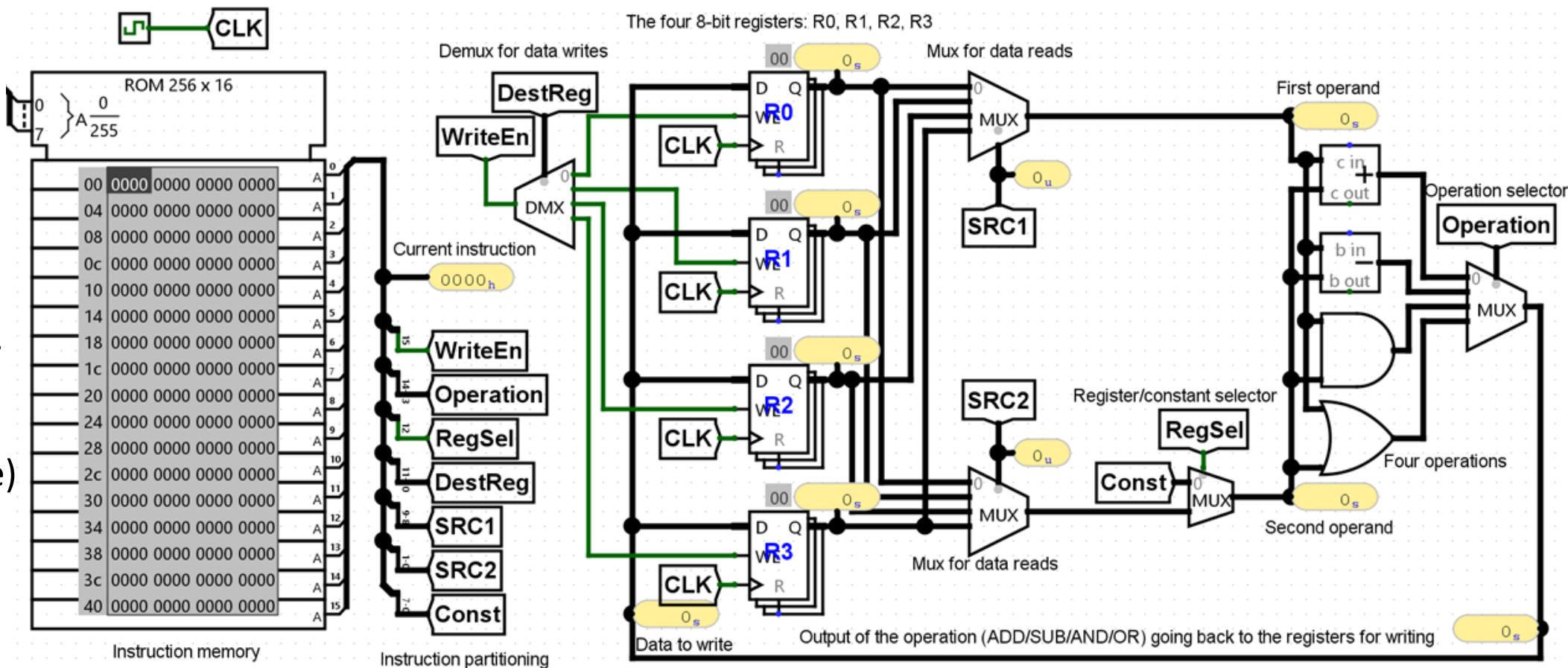
Processor implementation: Step 6

- If we want to run consecutive instructions, we have to **store them in a memory**

We have to provide an address.

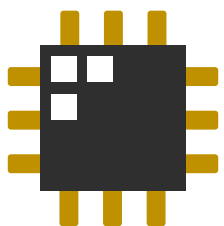
Every cycle:
 $\text{Addr} = \text{Addr} + 1$
(we move to the next instruction every positive edge)

What are we missing?



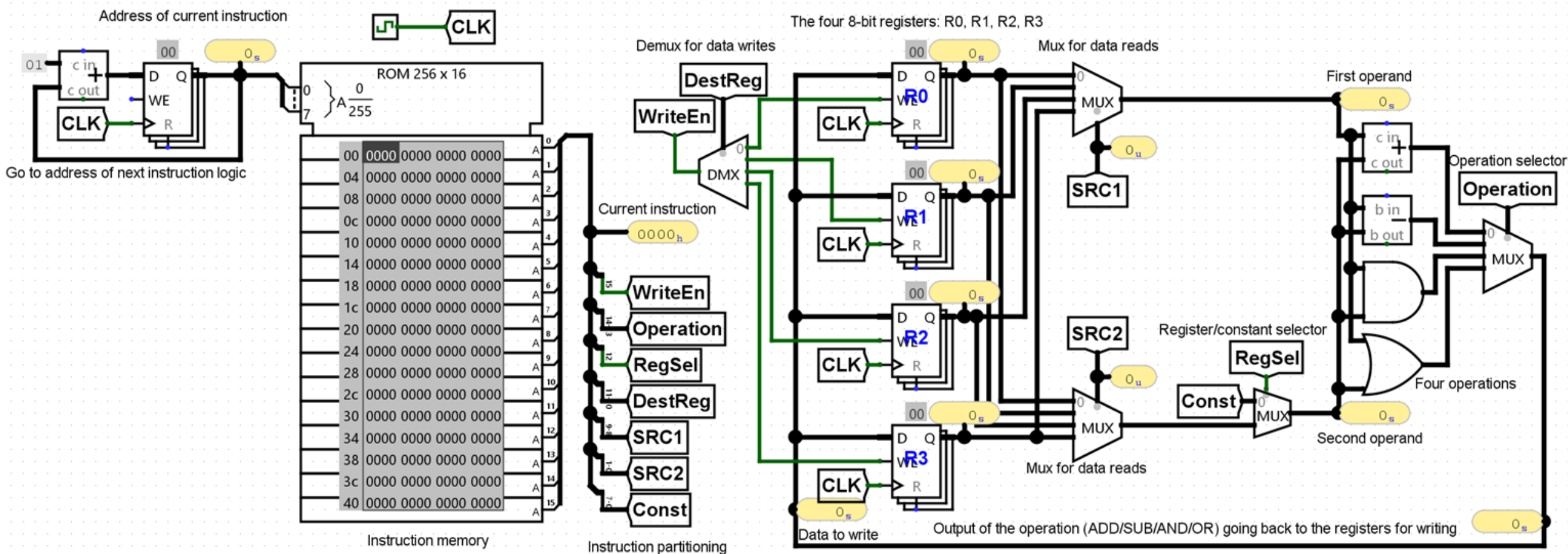


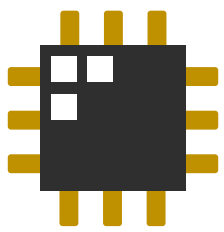
-
- Address of current instruction
- 01
- c in
- +
- c out
- 00
- 01
- CLK
- WE
- R
- Q
- Go to address of next instruction logic



Processor implementation: Version 2

This processor can now run consecutive instructions!





Playing around with instructions

- Try running these instructions on the processor:
 - You have to convert them to machine code first

ADDI R3, R0, 113

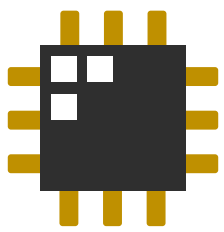
ADDI R2, R2, 21

SUB R1, R3, R2

ANDI R1, R1, 15

OR R0, R2, R1

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



Playing around with instructions

- Try running these instructions on the processor:
 - You have to convert them to machine code first

ADDI R3, R0, 113

ADDI R2, R2, 21

SUB R1, R3, R2

ANDI R1, R1, 15

OR R0, R2, R1

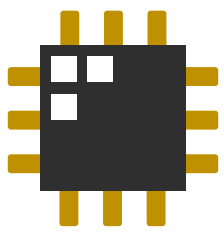
```
1 def convert_to_twos_complement(value, num_bits):
2     return bin((1 << num_bits) + value if value < 0 else value)[2:].zfill(num_bits)
3
4 def parse_instruction(instruction):
5     register_mapping = {'R0': '00', 'R1': '01', 'R2': '10', 'R3': '11'} # Register assignment for mux/demux selectors
6     operation_mapping = {'ADD': '00', 'SUB': '01', 'AND': '10', 'OR': '11'} # Operation assignment
7     condition_mapping = {'EQ': '00', 'NE': '01', 'LT': '10', 'GT': '11'} # Branch condition check for PC mux selector
8
9     parts = instruction.split()
10    write_en = str(int(not parts[0].startswith('B'))) # Always write unless it is a branch
11    operation = condition_mapping[parts[0].lstrip('B')] if parts[0].startswith('B') else operation_mapping[parts[0].rstrip('I')] # Extract the operation bits
12    const_sel = str(int(not (parts[0].endswith('I') or parts[0].startswith('B')))) # Everything that ends with I is an immediate
13    destreg = '00' if parts[0].startswith('B') else register_mapping[parts[1].rstrip(',')] # This is a don't care if it is a branch instruction actually
14
15    # If instruction is branch, the source register is the register at index 1 (destreg), else it is at index 2 (since dest register would be at index 1)
16    srcreg = register_mapping[parts[1].rstrip(',')] if parts[0].startswith('B') else register_mapping[parts[2].rstrip(',')]
17
18    if parts[0].endswith('I'): # If the instruction is an immediate type, convert to two's complement notation
19        const = convert_to_twos_complement(int(parts[3].rstrip(',')), 8)
20    elif parts[0].startswith('B'): # Branch instructions have their constants on index 2 instead of 3
21        const = convert_to_twos_complement(int(parts[2].rstrip(',')), 8)
22    else: # If the instruction does not have a constant, then it is a purely register-based instruction
23        const = '000000' + register_mapping[parts[3].rstrip(',')]
24
25    binary_result = write_en + operation + const_sel + destreg + srcreg + const # Combine all binary into 16-bit instruction
26    hex_result = hex(int(binary_result, 2))[2:].zfill(4).upper() # convert to hex
27    return hex_result
28 # End of actual assembler function. Less than 30 lines!
```

You can write an assembler to speed up the process.

^ This is the Python *assembler* for our processor.

Given the defined instruction encoding, we should be able to automate the process!

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



Playing around with instructions

- Try running these instructions on the processor:
 - You have to convert them to machine code first

ADDI R3, R0, 113

0x8C71

ADDI R2, R2, 21

0x8A15

SUB R1, R3, R2

0xB702

ANDI R1, R1, 15

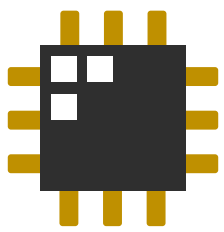
0xC50F

OR R0, R2, R1

0xF201

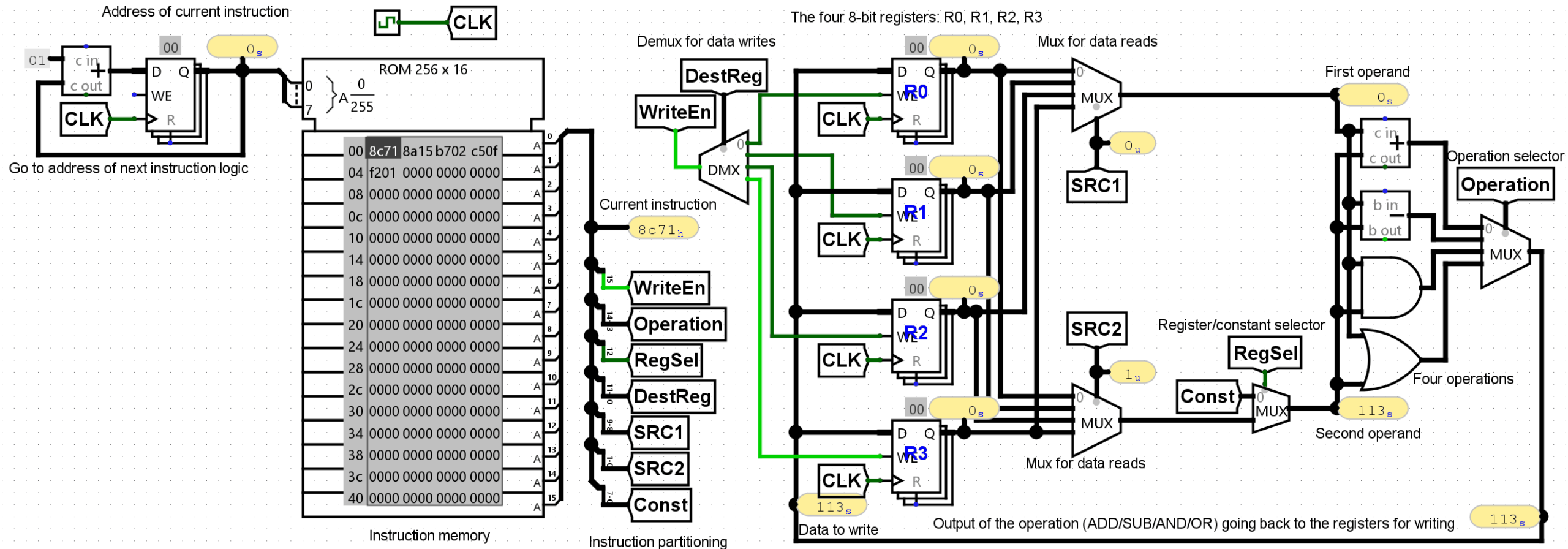
Given the defined instruction encoding, you can verify that our hexadecimal instruction codes are correct!

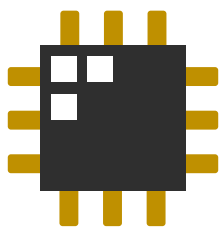
W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



Processor implementation: Version 2

Try it out! Run all 5 instructions one at a time!



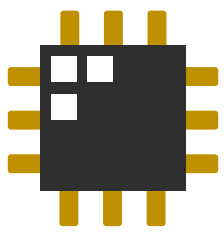


Supporting program loops

- Fun programs have loops!

```
r1 = 0
for r2 in range(5):
    r1 = r1 + 1
```

How do we do this on the processor?



Supporting program loops

- Fun programs have loops!

ADDI R2, R2, 5

(R2 = 5)

loop:

ADDI R1, R1, 1

(R1 = R1 + 1)

SUBI R2, R2, 1

(R2 = R2 - 1)

If R2 != 0, go back to **loop**

(how to do this?)

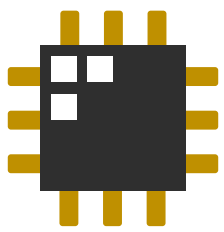
Note: The **loop** label is just a programming marker to where we want to go. It is not an instruction.

This one is *pointing* to the ADDI R1, R1, 1 instruction.

Instruction mastery questions:

How many times will we loop in this code?

What will be the final values of R1 and R2?



Supporting program loops

- **Branch*** instructions allow changing the current instruction address
 - You can add *conditions* on when to jump or not

ADDI R2, R2, 5

(R2 = 5)

loop:

ADDI R1, R1, 1

(R1 = R1 + 1)

SUBI R2, R2, 1

(R2 = R2 – 1)

BNE R2, -2

(If R2 $\neq$ 0, go back 2 steps)

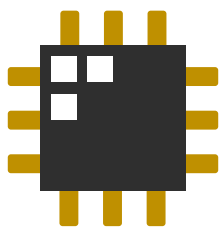
EQ (Equal to 0)

NE (Not Equal to 0)

LT (Less Than 0)

GT (Greater Than 0)

Branch is a computer architecture term. If you jump, you branch out of the normal program flow (one line after the other)



Supporting program loops

- How are branch instructions converted to machine code?

BNE R2, -2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
W	OPC	RS	DEST	SRC1	CONST/SRC2										

00 – EQ
01 – NE
10 – LT
11 – GT

0

00

00 – R0
01 – R1
10 – R2
11 – R3

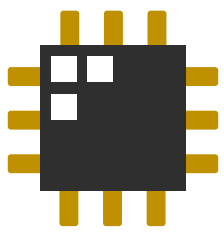
For branch instructions, use 8-bit CONST
This is written in two's complement

RS does not matter for branch instructions
(no second operand), set it to 0.

DEST does not matter for branch instructions
(it doesn't update the reg file), set it to 00.

This instruction does not update the register file: 0

--	--	--	--	--	--



Supporting program loops

- How are branch instructions converted to machine code?

BNE R2, -2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
W	OPC	RS	DEST	SRC1	CONST/SRC2										

00 – EQ
01 – NE
10 – LT
11 – GT

0

00

00 – R0
01 – R1
10 – R2
11 – R3

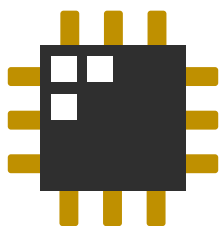
For branch instructions, use 8-bit CONST
This is written in two's complement

RS does not matter for branch instructions
(no second operand), set it to 0.

DEST does not matter for branch instructions
(it doesn't update the reg file), set it to 00.

This instruction does not update the register file: 0

0	01	0	00	10	1111 1110									
---	----	---	----	----	-----------	--	--	--	--	--	--	--	--	--



Instruction encoding summary

- We're increasing in complexity! (*This is the last one, promise*)

W	OPC	RS	DEST	SRC1	CONST/SRC2
↓	If W = 0: 00 – EQ 01 – NE 10 – LT 11 – GT	↓	If W = 1: 00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	If W = 1: 00 – ADD 01 – SUB 10 – AND 11 – OR		If W = 0: 00 – branches do not write		If RS = 1: Second operand is the last 2-bits SRC2 0000 00 <u>00</u> – R0 0000 00 <u>01</u> – R1 0000 00 <u>10</u> – R2 0000 00 <u>11</u> – R3
			If W = 0: 0 – branches have no second operand		
			If W = 1: 0 – Constant second operand (ADDI/SUBI/ANDI/ORI) 1 – Register second operand (ADD/SUB/AND/OR)		

Sample instructions:

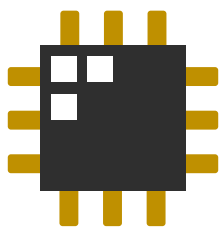
ADD R0, R2, R3

SUBI R3, R1, -31

BNE R2, -2

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

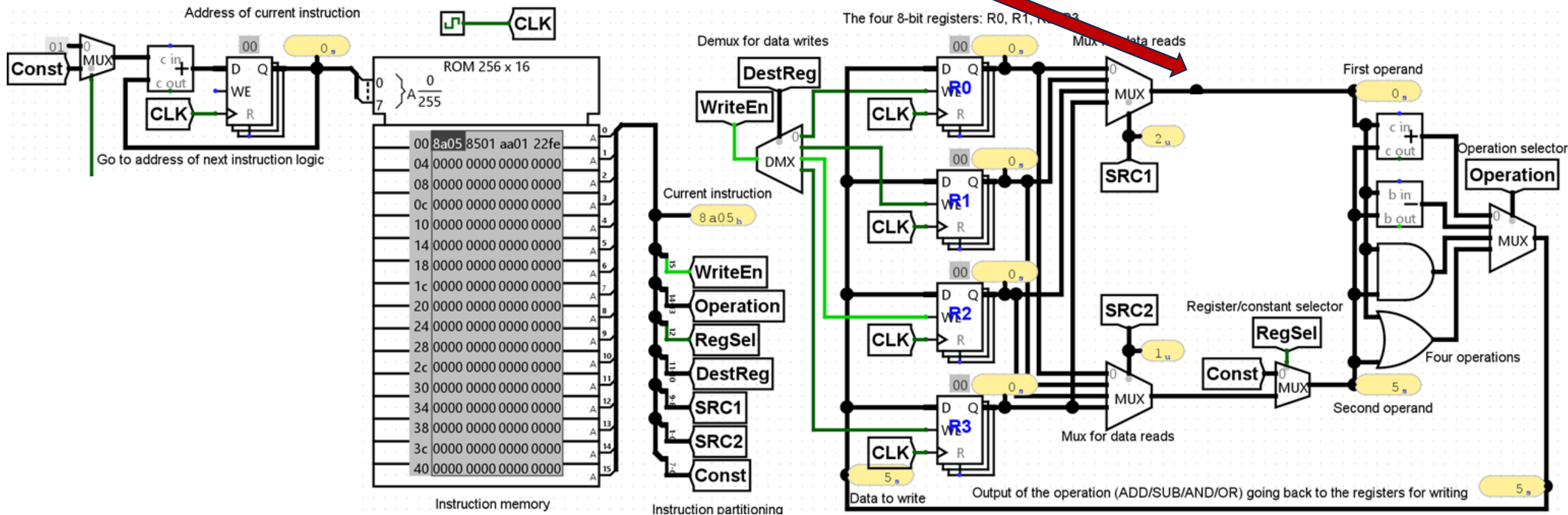
1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)

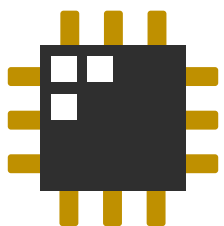


Processor implementation: Step 8

Branch instructions compare the SRC1 value (the first operand) relative to 0

What are we missing?

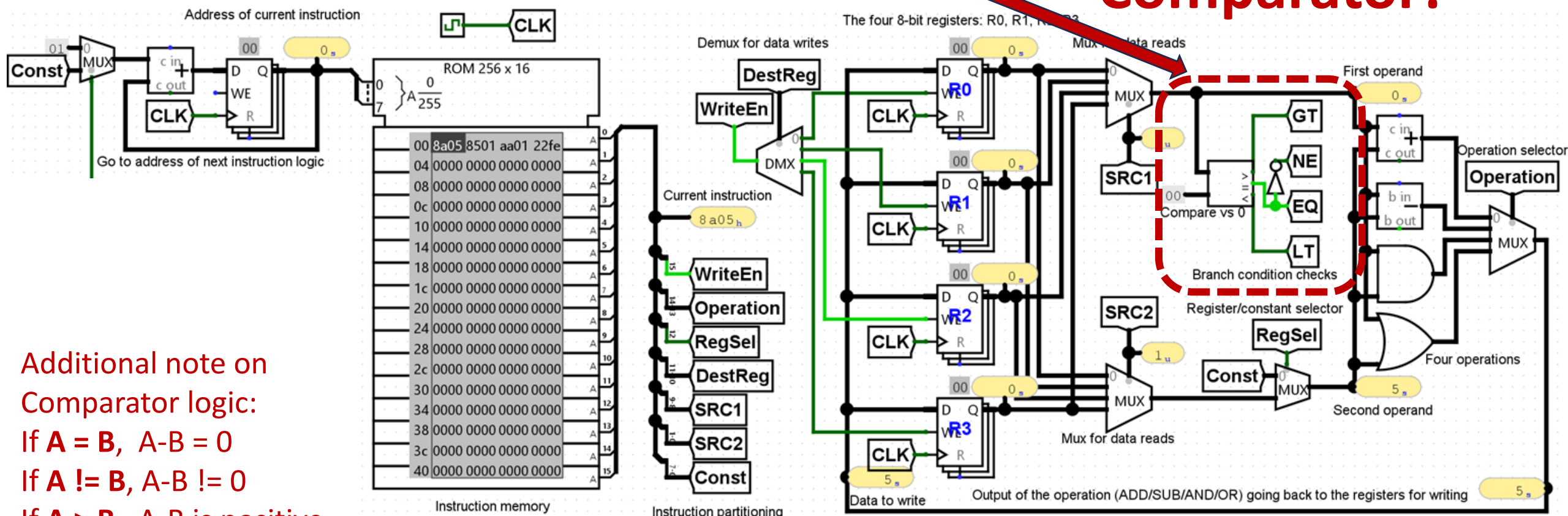




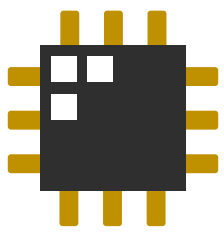
Processor implementation: Step 8

Branch instructions compare the SRC1 value (the first operand) relative to 0

Comparator!

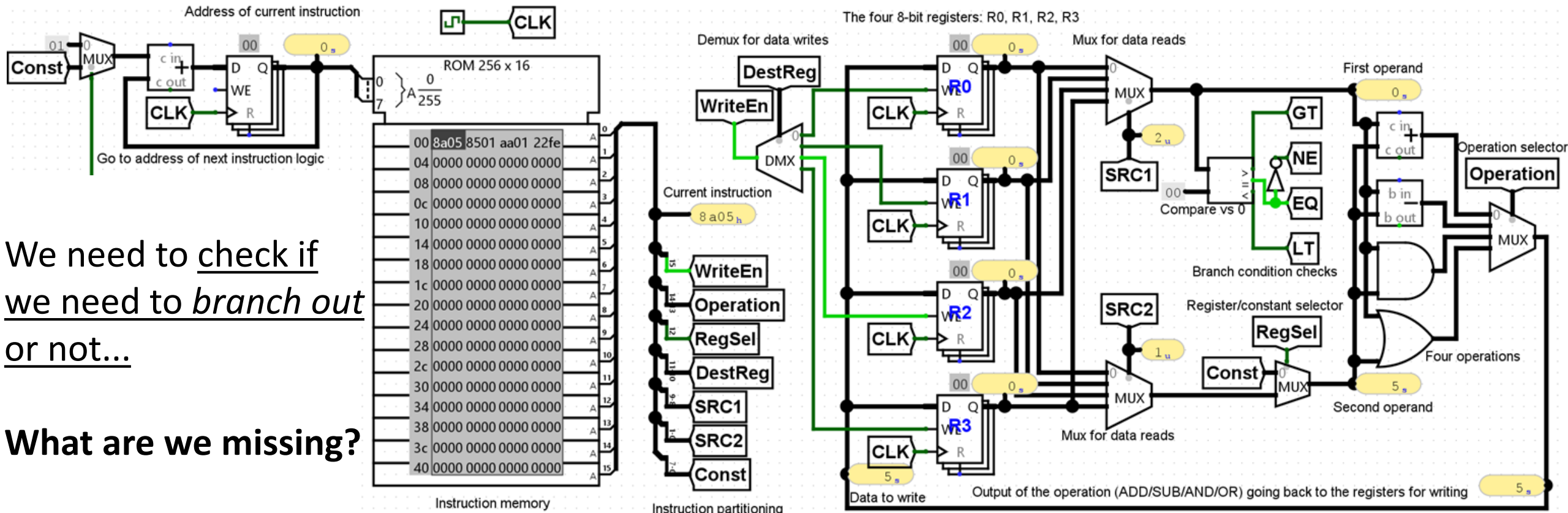


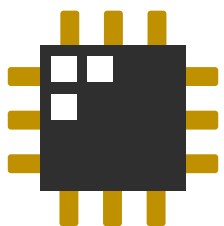
Note: Comparators are just subtractors inside, allowing us to compare two operand values



Processor implementation: Step 8

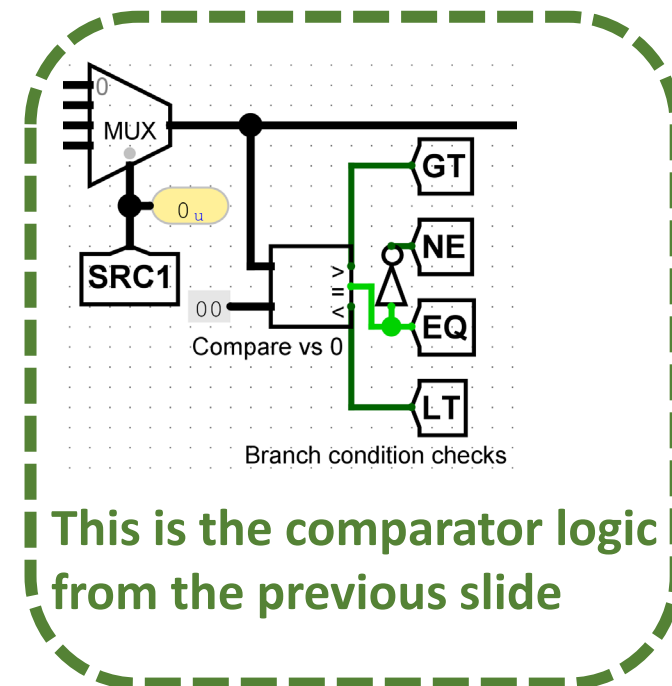
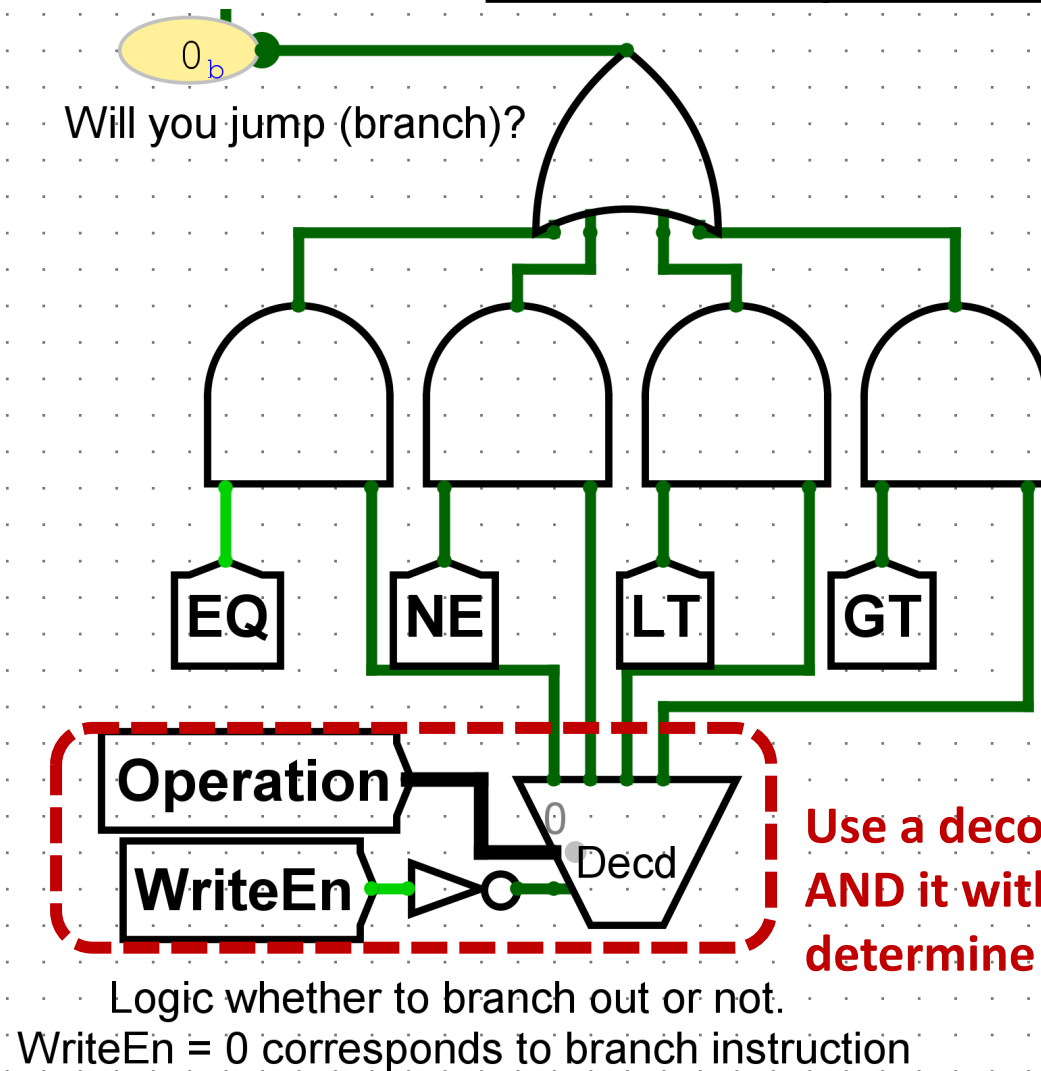
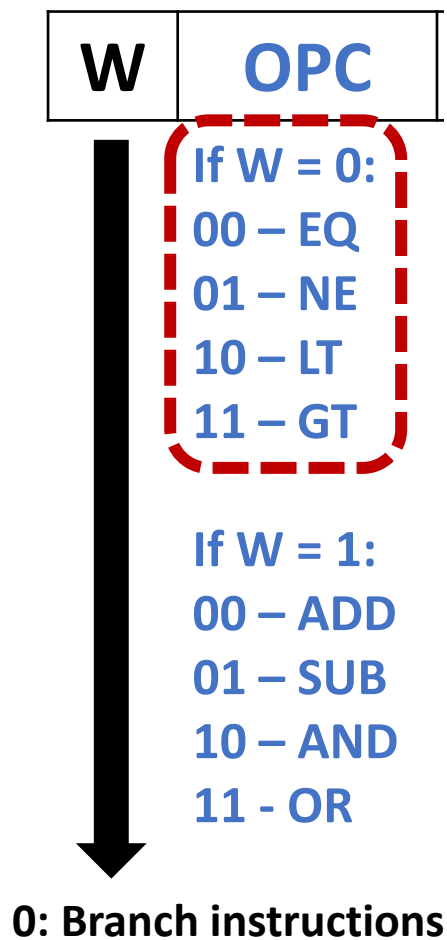
Branch instructions compare the SRC1 value (the first operand) relative to 0



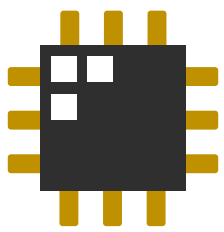


Processor implementation: Step 9

We check for the operation code to determine if the condition is satisfied



Use a decoder to identify the branch condition. AND it with the comparator outputs to determine if you're branching out or not.



Processor implementation: Step 10

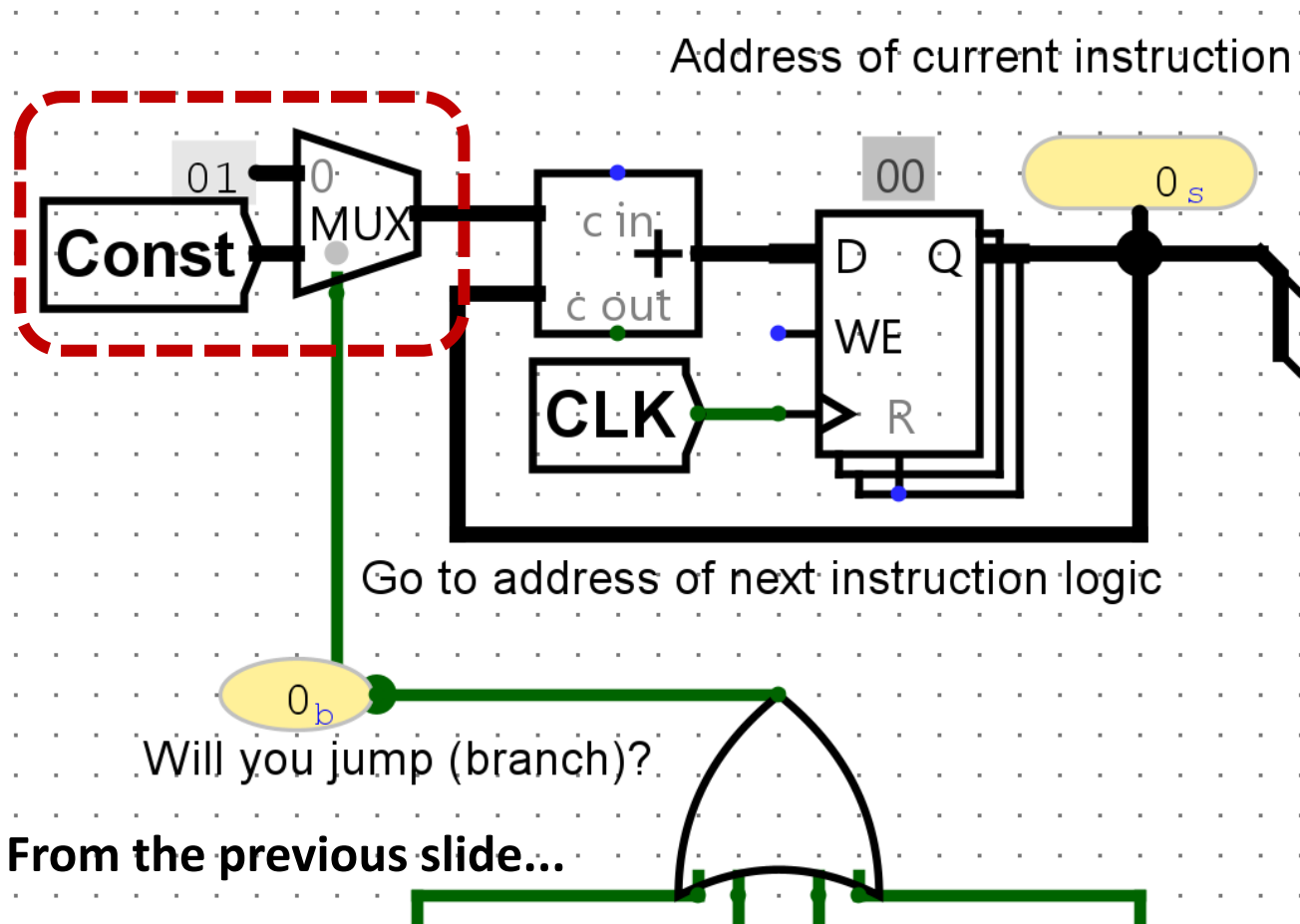
If the condition is satisfied, we have to branch out to a new address

We have to modify the address counter to add a constant instead of just +1 when branching out

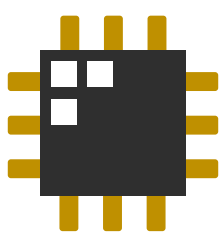
Recall the example:

BNE R2, -2

This means that if the condition is true, our new address is -2 from where we are right now (-2 is a constant in the instruction)

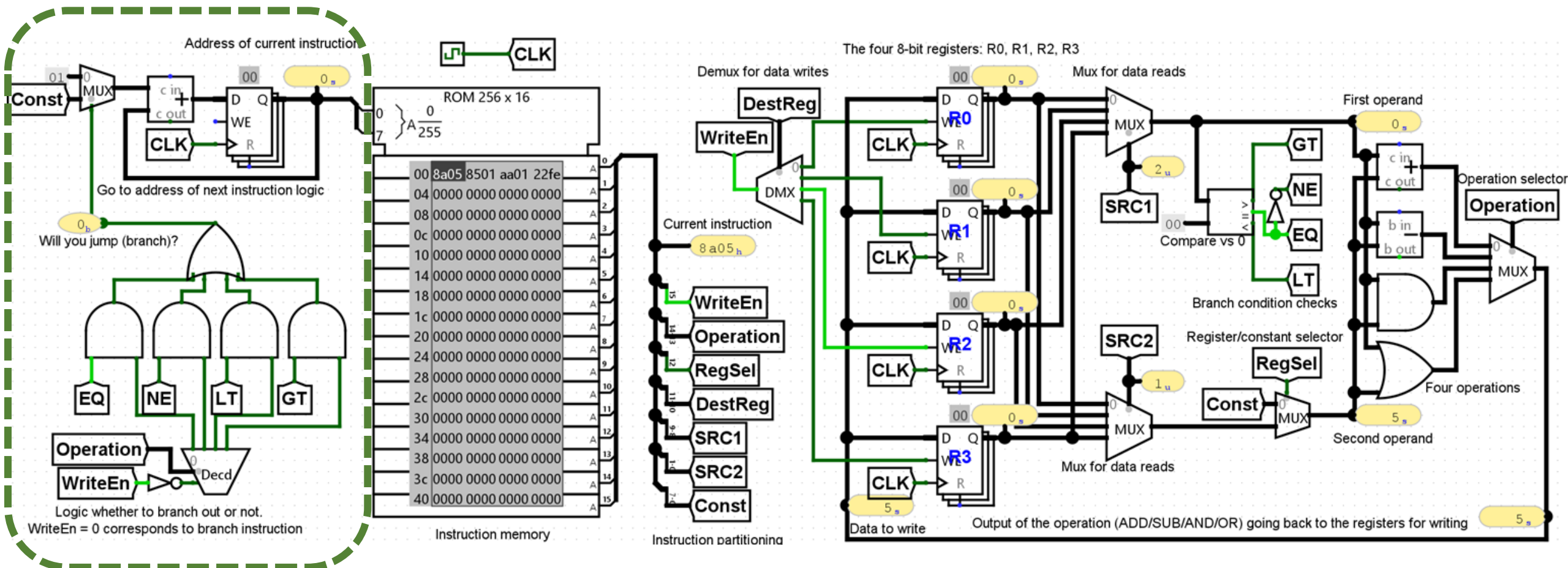


From the previous slide...

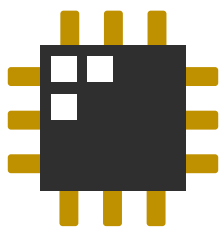


Processor implementation: Step 10

Branch instructions allow us to go to a different address if the condition is true

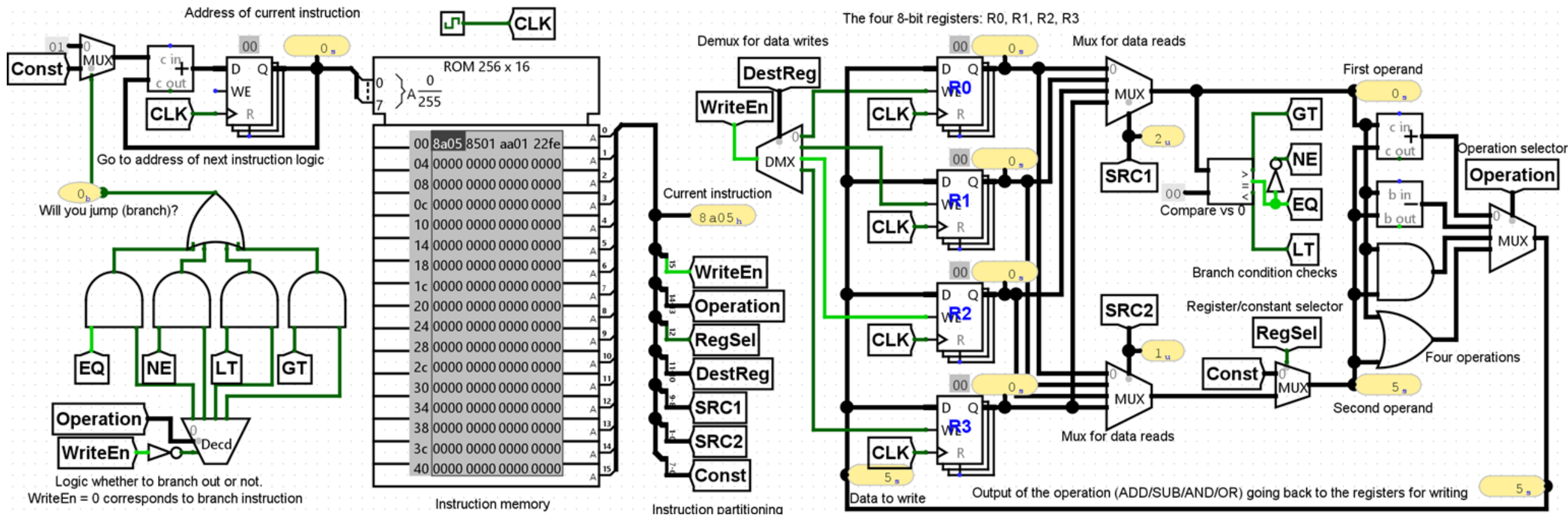


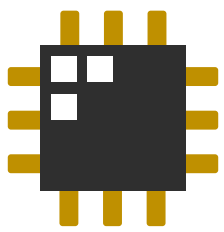
This is the branchdecoder logic from previous two slides



Processor implementation: Final Version!

This processor can now handle branch instructions as well!





Instruction encoding summary

- Recap: The instruction encoding that the processor supports

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

If W = 0:

00 – EQ

01 – NE

10 – LT

11 – GT

If W = 1:

00 – ADD

01 – SUB

10 – AND

11 – OR

If W = 1:

00 – R0

01 – R1

10 – R2

11 – R3

If W = 0:

00 – branches do not write

If W = 0:

0 – branches have no second operand

If W = 1:

0 – Constant second operand (ADDI/SUBI/ANDI/ORI)

1 – Register second operand (ADD/SUB/AND/OR)

00 – R0

01 – R1

10 – R2

11 – R3

If RS = 0: The second operand is 8-bit CONST
This is written in two's complement

If RS = 1: Second operand is the last 2-bits SRC2

0000 0000 – R0

0000 0001 – R1

0000 0010 – R2

0000 0011 – R3

Sample instructions:

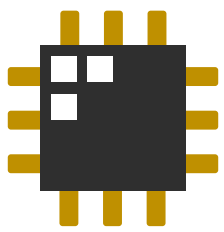
ADD R0, R2, R3

SUBI R3, R1, -31

BNE R2, -2

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)



Sample program execution with loops!

- Let's run this now!

ADDI R2, R2, 5

0x8A05

loop:

(this is just a label)

ADDI R1, R1, 1

0x8501

SUBI R2, R2, 1

0xAA01

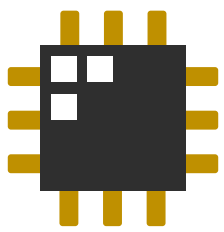
BNE R2, -2

0x22FE

The **loop** label is only relevant when writing the instructions. It is just a programming marker to where we want to jump into if the branch condition is true. It has no machine code equivalent since it is not an instruction.

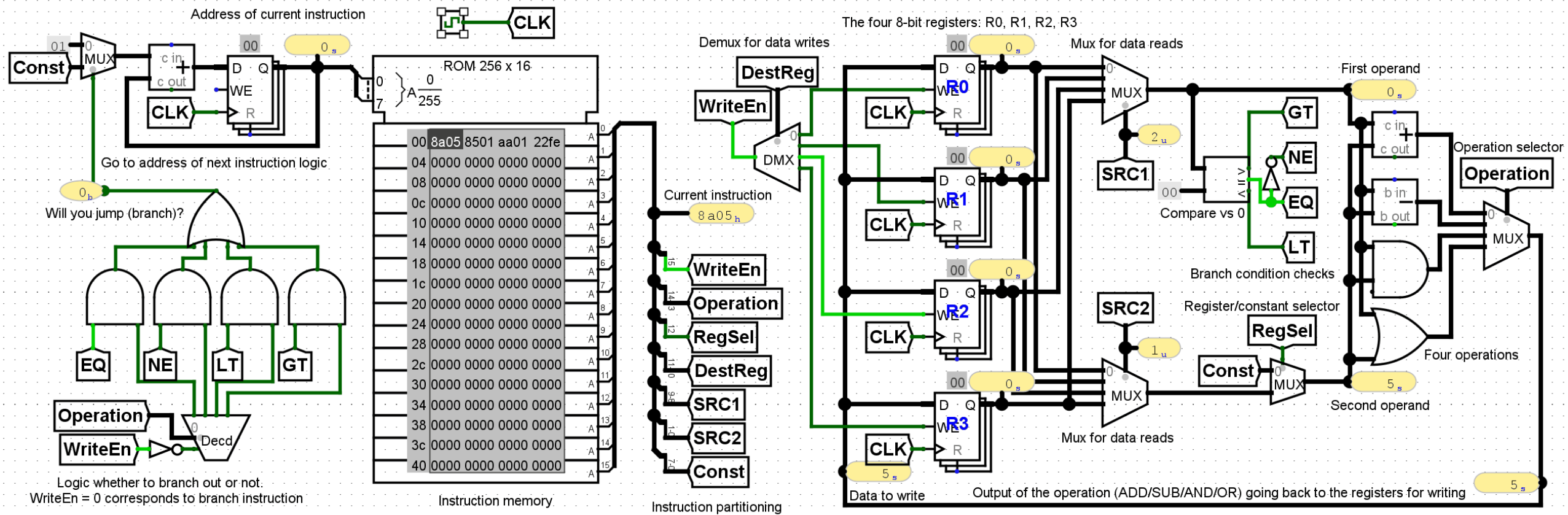
The **loop** label is technically marking where the ADDI R1, R1, 1 instruction is (the first instruction below it).

Relative to the BNE instruction, ADDI R1, R1, 1 is -2 instructions from it. (Negative since it goes back 2 steps)



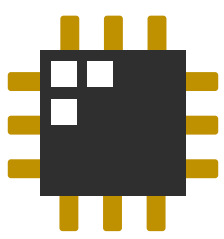
Sample program execution with loops!

• Let's run this now!



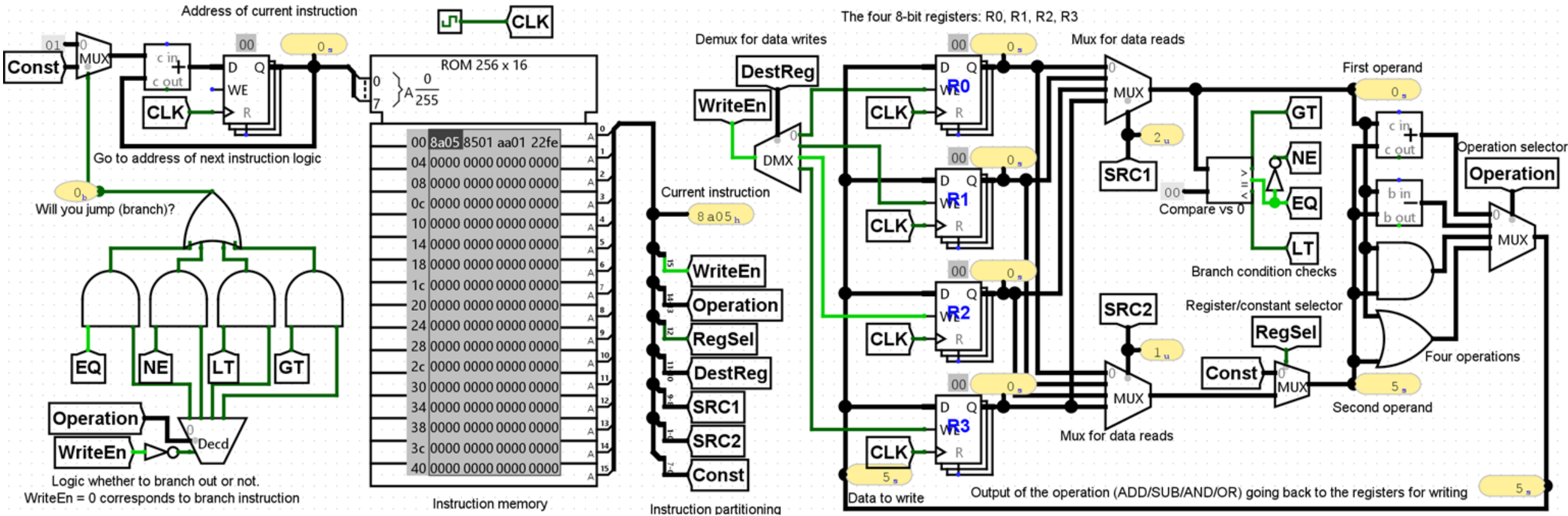
Note how the address goes back to '1' (Instruction 0x8501, which is the ADDI R1, R1, 1 instruction) after address '3' (Instruction 0x22fe, which is the BNE R2, -2 instruction). That's the loop in action!

Check the values of R1 and R2 as we go through the loop. Are they correct every step of the way?



Your first processor ☺

- 16-bit instructions, 8-bit data, 4 registers, 4 operations, 4 branch conditions



This is a *custom processor* made for EEE 113. Simple enough to be understood. Complex enough to implement simple programs. Previous EEE 113 offerings had 'Machine Architecture that is Really Intuitive and Easy' (MARIE) Now, for this offering, we have 'Architecture Design that's Easy to Learn' (ADEL) ☺